

Intelligent System for Plant Disease Detection and Remedy Recommendation

A major project report submitted in partial fulfilment of the requirement
for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted by

Kushal Sharma (221030116)

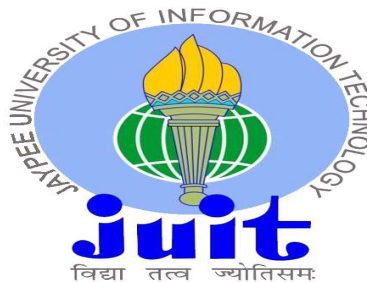
Mehak Singhal (221030297)

Harsha Thakur (221030461)

Devansh Moudgil (221030465)

Under the guidance & supervision of

Ms. Vani Sharma



**Department of Computer Science & Engineering and
Information Technology**

Jaypee University of Information Technology,

Waknaghat, Solan - 173234 (India)

May 2026

Certificate

This is to certify that the major project report entitled “**Intelligent System for Plant Disease Detection and Remedy Recommendation**”, submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is a bonafide project work carried out under my supervision during the period from July 2025 to May 2026.

We have personally supervised the research work and confirm that it meets the standards required for submission. The project work has been conducted in accordance with ethical guidelines, and the matter embodied in the report has not been submitted elsewhere for the award of any other degree or diploma.

Date:
Place: JUIT, Solan

(Supervisor Signature)
Supervisor Name: Ms. Vani Sharma
Designation: Assistant Professor (Grade-I)
Department: CSE & IT

Declaration

We hereby declare that the work presented in this major project report entitled “**Intelligent System for Plant Disease Detection and Remedy Recommendation**”, submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is an authentic record of our own work carried out during the period from July 2025 to May 2026 under the supervision of Ms. Vani Sharma

We further declare that the matter embodied in this report has not been submitted for the award of any other degree or diploma at any other university or institution.

(Student Signature)
Name: Kushal Sharma
Roll No: 221030116
Date:

(Student Signature)
Name: Mehak Singhal
Roll No: 221030297
Date:

(Student Signature)
Name: Harsha Thakur
Roll No: 221030461
Date:

(Student Signature)
Name: Devansh Moudgil
Roll No: 221030465
Date:

This is to certify that the above statement made by the candidates is true to the best of my knowledge.

Date:
Place: JUIT, Solan

(Supervisor Signature)
Supervisor Name: Ms. Vani Sharma
Designation: Assistant Professor (Grade-I)
Department: CSE & IT

ACKNOWLEDGEMENT

Firstly, we express our heartiest thanks and gratefulness to almighty God for the divine blessing that made the successful completion of this project possible.

We are grateful and express our profound indebtedness and sincere thanks to our supervisor Ms. Vani Sharma, Assistant Professor (Grade-I) in the Department of Computer Science and Engineering at Jaypee University of Information Technology, Wagnaghat. We are deeply grateful for her her scholarly guidance, endless patience, and constant encouragement throughout the duration of this work. Her keen interest, constructive criticism, and valuable advice at every stage from refining initial drafts to final execution were instrumental in bringing this project to fruition.

We would also like to extend our thanks to the entire faculty and staff of the Department of CSE for their consistent support and for providing the necessary facilities to conduct our research.

Our thanks also go to all the individuals who helped us, directly or indirectly, in making this project a success. The timely assistance and cooperation from both teaching and non-teaching staff members have been greatly appreciated. .

Finally, we acknowledge with due respect the constant support and patience of our parents.

Student Name & (Roll No.)
Kushal Sharma (221030116)

Student Name & (Roll No.)
Mehak Singhal (221030297)

Student Name & (Roll No.)
Harsha Thakur(221030461)

Student Name & (Roll No.)
Devansh Moudgil (221030465)

TABLE OF CONTENTS

LIST OF TABLES	i
LIST OF FIGURES	ii-iv
LIST OF ABBREVIATIONS & NOMENCLATURE.	v-vi
ABSTRACT	vii
1. INTRODUCTION.....	1-5
1.1 Introduction.....	1
1.2 Problem Statement.....	2
1.3 Objective.....	3
1.4 Significance and motivation of the project report.....	4
1.5 Organization of project report.....	5
2. LITERATURE SURVEY.....	6-13
2.1 Overview of relevant literature.....	6-13
2.2 Key gaps in the literature.....	13
3. SYSTEM DEVELOPMENT.....	14-43
3.1 Requirements and Analysis.....	14
3.1.1 Functional Requirements.....	14
3.1.2 Non-Functional Requirements.....	15
3.1.3 Technical Analysis.....	15
3.2 Project Design and Architecture.....	16-19
3.3 Data Preparation.....	19
3.3.1 Data Merging and Cleaning.....	20

3.3.2 Image Standardization.....	20
3.3.3 Train - Validation Split.....	20
3.3.4 Data Augmentation Strategy.....	21
3.3.5 Class Balancing.....	21
3.4 Implementation.....	22
3.4.1 System Architecture Overview.....	23
3.4.2 Tools and Technologies Used.....	23-24
3.4.3 Implementation of eight deep learning models.....	24-39
3.4.4 Unified Training Pipeline	39
3.5 Key Challenges.....	39-43
4. TESTING AND EVALUATION.....	44-49
4.1 Testing Strategy.....	44
4.1.1 Unit Testing.....	44-45
4.1.2 Integration Testing.....	45
4.1.3 System Testing.....	45-46
4.1.4 User Acceptance (UAT).....	46
4.1.5 Performance Testing.....	46
4.2 Test Case and Results.....	47-49
5. RESULTS AND EVALUATION.....	50-67
5.1 Results.....	50-65
5.1.1 U-Net	51
5.1.2 Vgg-16	51
5.1.3 ViT	52

5.1.4 Vgg-19	52
5.1.5 AlexNet	55
5.1.6 DenseNet	57
5.1.7 ResNet.....	58
5.1.8 EfficientNet-B4	59
5.2 Comparison and Existing Solutions.....	66
5.3 Comparison of Various Trained Models.....	67
6. CONCLUSIONS AND FUTURE SCOPE.....	68-71
6.1 Conclusion.....	68-69
6.2 Future Scope.....	70-71
REFERENCES.....	72-73

LIST OF TABLES

Table No.	Description	Page No.
2.1	Summary of Literature Survey	7-13
2.2	Class Balancing Parameters	22
2.3	Tools and technologies used for implementing models	23-24
5.3	Comparison of all trained models based on key performance metrics.	67

TABLE OF FIGURES

S. No.	Figure No.	Title	Page No.
1	Figure 3.1	High-Level System Architecture	16
2	Figure 3.2	Client Application Diagram	17
3	Figure 3.3	Backend Server Diagram	18
4	Figure 3.4	Sequence Diagram – Real-Time Synchronisation	19
5	Figure 3.5	Implementation of UNet Segmentation Module	25
6	Figure 3.6	UNet Architecture	25
7	Figure 3.7	Dice Loss	26
8	Figure 3.8	Training Loop	26
9	Figure 3.9	Loading the ResNet50 model	26
10	Figure 3.10	Compile the model	27
11	Figure 3.11	Loading ViT model	27
12	Figure 3.12	Transfer learning strategy for ViT model	28
13	Figure 3.13	Training step with mixed precision	28
14	Figure 3.14	Fine tuning stage	28
15	Figure 3.15	Preparing the VGG16 model	29
16	Figure 3.16	Training the model	29

17	Figure 3.17	Running the predictions	30
18	Figure 3.18	Confusion Matrix	30
19	Figure 3.19	ROC Curve	31
20	Figure 3.20	Evaluation Metrics	31
21	Figure 3.21	Loading the Data	32
22	Figure 3.22	Training Vgg-19 Model	32
23	Figure 3.23	Data Validation	33
24	Figure 3.24	Loading model weights	33
25	Figure 3.25	Loading Alexnet model	34
26	Figure 3.26	Building Alexnet model like CNN	34
27	Figure 3.27	Training the model	35
28	Figure 3.28	Loading DenseNet121	35
29	Figure 3.29	Training the model	36
30	Figure 3.30	Training Configuration	36
31	Figure 3.31	Data Augmentation	37
32	Figure 3.32	Load Dataset	37
33	Figure 3.33	Model Training	38
34	Figure 3.34	Model Validation	38
35	Figure 3.35	Test Time Augmentation	39
36	Figure 5.1	UNet	51

37	Figure 5.2	Vgg-16	52
38	Figure 5.3	Vit Classification Report	52
39	Figure 5.4	Vgg-19 Confusion Matrix	53
40	Figure 5.5	Vgg-19 ROC curve	53
41	Figure 5.6	Vgg-19	54
42	Figure 5.7	AlexNet Accuracy graph	55
43	Figure 5.8	AlexNet Loss graph	56
44	Figure 5.9	AlexNet Confusion Matrix	56
45	Figure 5.10	Alexnet Classification Report	57
46	Figure 5.11	DenseNet Core Metrics	57
47	Figure 5.12	DenseNet Learning curve Analysis	58
48	Figure 5.13	DenseNet Advanced Metrix	58
49	Figure 5.14	ResNet	58
50	Figure 5.15	EfficientNet-B4 Comparative Results	59
51	Figure 5.16	EfficientNet-B4 Best Checkpoint	59
52	Figure 5.17	EfficientNet-B4 Lab Performance	59
53	Figure 5.18	EfficientNet-B4 Field Results	60
54	Figure 5.19	EfficientNet-B4 Classification Report	60
55	Figure 5.20	EfficientNet-B4 Final Summary	61

56	Figure 5.21	EfficientNet-B4 Confusion Matrix	61
57	Figure 5.22	EfficientNet-B4 ROC Curve	61
58	Figure 5.23	EfficientNet-B4 Loss Curve	62
59	Figure 5.24	EfficientNet-B4 Accuracy Curve	62
60	Figure 5.25	EfficientNet-B4 Learning Rate Curve	62
61	Figure 5.26	PlantCare Home Page	63
62	Figure 5.27	PlantCare Diagnose Page	63
63	Figure 5.28	Apple Leaf Image	63
64	Figure 5.29	PlantCare Analysis	64
65	Figure 5.30	User Upload Image Interface	65
66	Figure 5.31	Apple Leaf Overall Results	65

LIST OF ABBREVIATIONS, SYMBOLS OR NOMENCLATURE

ABBREVIATIONS

S. No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CNN	Convolutional Neural Network
4	ResNet	Residual Neural Network
5	CSS	Cascading Style Sheets
6	ViT	Vision Transformer
7	HTML	Hyper Text Markup Language
8	DL	Deep Learning
9	JS	JavaScript
10	ML	Machine Learning
11	FLASK	Python Micro Web Framework
12	ReLU	Rectified Linear Unit
13	UI	User Interface
14	UX	User Experience

NOMENCLATURE

S. No.	Term	Definition
1	TensorFlow/ Keras	Deep learning frameworks employed to train and execute disease detection models.
2	PlantVillage Dataset	Benchmark dataset containing ~54,000 labeled leaf images for 38 crop-disease classes.
3	U-Net	CNN-based model mostly applied for image segmentation, e.g., marking diseased leaf areas.
4	ResNet50	A deep residual learning framework employed for high-accuracy image classification.
5	Vision Transformer (ViT)	A transformer-based framework adapted for image classification tasks.
6	Flask	Lightweight Python framework employed to construct the backend API for disease prediction.
7	Augmentation	Methods such as rotation, flipping, and zooming used on training data to enhance model robustness.
8	Confidence Score	Probability given by the model to represent how confident it is about the predicted disease label.

ABSTRACT

Plant diseases remain one of the main causes of low yield among most agricultural products. Early and accurate diagnosis is of importance in mitigating losses; however, most existing methods solely rely on manual inspection and expert knowledge, usually time-consuming, inconsistent, and inaccessible to rural farmers. Recent progress in artificial intelligence, especially computer vision and deep learning, opens new perspectives in this challenge by making possible the automated detection of plant-leaf diseases from images.

This project presents a web-based intelligent system that detects plant diseases and recommends remedies. With the use of deep learning models like U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4 the system tries to classify leaf images as either healthy or diseased. These models mentioned earlier have been trained and evaluated on the PlantVillage and Plant Doc dataset using different evaluation metrics. The PlantVillage dataset contains 38 crops and disease classes. The PlantVillage dataset is merged with the Plant DOC dataset to contain a total of 57,000 images. Augmentation of data had been done for the sole purpose of generalization. After a disease has been discovered by the system remedies are given at that time only to bridge the gap of query and response.

This system enables an accurate and a user-friendly solution for farmers to their day-to-day identification and cure of diseased crops. The system also tries to reduce crop losses and improve yield by reducing the dependencies on manual sense.

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

Agriculture is one of the important sectors that provide human nutrition and a healthy lifestyle [1]. It remains a main source of livelihood for several people in India. Being an agricultural economy, a huge loss is faced by farmers when disease hits their crops. Delay in detecting the disease and lack of knowledge base for remedy results in a significant amount of yield loss which in turn affects their finances [2].

Traditional people in villages detect the crop status visually. Sometimes they may be right to detect diseases but experience may not always be right. This way is slow, inaccurate and not scientific [3]. Applying these old similar patterns to detect disease lowers the possible treatment of crops. They also cannot take advice from experts in this field as they are costly to approach and there is a lack of such people in rural farming communities [4]. All these shortcomings propose a system that is fast, accurate and accessible to each one of them.

Its solution lies among artificial intelligence innovations in computer vision and deep learning [4]. In comparison to standard models, Vision Transformers and Convolutional Neural Networks identify fine features like shift in color, change in texture and disease areas on leaves with greater accuracy [5]. If they used these tools and made them accessible digitally, they could give farmers instant disease discovery.

The project of this report tries to build an intelligent web-based system for detection and discovery of plant disease. The user uploads a leaf image on the web interface, Using deep learning models such as U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4, the system classifies it as healthy or not healthy. In addition to this detection, the system also provides required remedy suggestions to treat the disease. This helps in reducing the loss of crops, increasing awareness of right measures to be taken by farmers and increasing their productivity.

1.2 PROBLEM STATEMENT

Every year thousands of crops are disease hit, decreasing the net yield for producers as well as consumers. In rural areas, farmers are dependent on their local knowledge for checking the disease in crops. This is not reliable, accurate and subject to errors. By the time the discovery of disease is made, the damage has already been done and all the crops are wasted leading to huge produce as well as financial loss.

Limitations in existing systems for detecting plant disease and management are summarized below:

- Manual dependence: Most disease detection ways are based on human knowledge that includes experts who are unavailable or most probably absent in villages.
- Standard image processing tools are not capable enough to capture variations in symptoms which leads to wrong discovery or failure in early stages of disease.
- Fragmented tools: Some tools do detect the disease but do not recommend remedy while others provide static information with no real time updates. In fact, farmers themselves have to toggle between different platforms for diagnosis, advice, and guidance on treatment.
- Scalability Issues: Most of the existing research models present high accuracy in a lab, but consume a great deal of computational resources and hence cannot be used at larger scales or field levels.
- Less Intelligent Guidance: There exist very few platforms containing the art of using deep learning models that can handle variable datasets across different crops. Most of them do not have an overall coverage of crops and also lack artificial intelligence based support to guide farmers with valuable results.

Therefore, there exists an important need for a united, collective, intelligent, and a single system that is not only able to detect plant diseases with high accuracy but is also able to recommend the desired remedies.

1.3 OBJECTIVES

The main aim of our project is to build a web-based user interface that can help farmers quickly find out if their plants are infected. Many of these methods used till date right now are either slow, less accurate, or not easy for farmers to use and understand.

This project helps to solve the problems. The main points are given below:

- **Automation of Disease Detection:** The system should be able to look at an image of a leaf and provides the detected disease present and its suggestible remedy. For this we used models like U-Net, ResNet50, ViT, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4 to make this possible.
- **Classification with High Accuracy:** All of the eight models trained will be tested and we accept the model that gives the best and most reliable results for various crops and diseases.
- **Suggestable Remedy Recommendations:** Once our system finds a disease in plant leaves, it should give us an easy remedy suitable which could be applied on crops.
- **Data Preprocessing and Augmentation:** Before training the model, the images will be adjusted and cleaned in order to give accurate results which includes flipping, scaling and rotating the images.
- **User-Friendly Web Platform:** The website should be easy to use so that, having any technical skills, farmers can easily upload images and use it.
- **Scalable Deployment:** The system should not hang if many users are using it at the same time. It should work smoothly.
- **Contribution to Sustainable Agriculture:** Telling farmers early about disease and its remedy can help reduce crop loss leading to better harvests and promote sustainable farming activities among them.

1.4 SIGNIFICANCE AND MOTIVATION FOR THE PROJECT

The importance of this project lies in daily problems the farmers have to go through to feed the people. The losses of crops are not due to lack of efforts by farmers but due to late discovery of diseases and unknown remedies. Depending on some expert which can help them is very expensive and does not provide a constant touch of availability for farmers to solve their problem. We aim to solve it by handing them a web interface accessible on their phones with an internet connection and a camera to click an image of a leaf.

The reasons that motivate us to take this project are given below:

- **Crop Loss Reduction:** Farmers work for months but a single outbreak of disease destroys their crops and clears their earning. We aim to help them by giving a tool to spot early diseases and act according to the remedy provided by the system.
- **Technology for All Farmers:** Access to agricultural experts is often limited, especially in rural areas. Even a user with a normal smartphone can access this web interface. This system is designed to be accessible and useful for all users, regardless of technical expertise.
- **Supporting Rural Communities:** Most people who are affected by crop losses are farmers in rural areas. This system will provide them help in fast and accurate assistance with improved yields and a stable income.
- **Sustainable Farming Practices:** Heavy use of pesticides which are not necessarily needed can be stopped if disease is discovered earlier. It ensures healthy farming and is environment friendly. Also is less costly.
- **Applying AI:** What we get excited about, when it comes to AI, is actually applying the technology outside of theory and research papers. In fact, using models like ResNet50 and the Vision Transformers, we can actually implement a system to solve a real-world problem in fields

At its most basic, the impetus for this project is straightforward: we want to minimize the challenges facing farmers by providing them with a real, smart, and user-friendly tool to aid them in defending their crops and incomes.

1.5 PROJECT REPORT ORGANISATION

This project report is organised into three major chapters for simplicity of comprehension.

Chapter 1: Introduction

This chapter starts the project by defining the project background, problem statement, objectives, and significance and motivation of the project. It concludes with the description of the structure of the project report.

- Introduction
- Problem Statement
- Objectives
- Significance and Motivation of the Project Work
- Organization of Project Report

Chapter 2: Literature Review

Recent developments will be highlighted in the literature review in this chapter, and such a comprehensive review of the literature has not been undertaken as a reference. It also speaks of significant flaws in our current knowledge and outlines what the project will hopefully do.

- Overview of Relevant Literature
- Key Gaps in the Literature

Chapter 3: 3.1-System Development – This chapter describes how the system has been planned and built. It covers the requirements, design, and architecture of the project, as well as the technologies used for development.

CHAPTER 2: LITERATURE REVIEW

2.1 OVERVIEW OF RELEVANT LITERATURE

We study in our research important areas for creating an intelligent system for detecting plant diseases as well as suggesting remedies. The literature offers a solid background in plant image analysis, deep learning models for detecting diseases, and practical approaches for designing a powerful and accurate AI-based system.

2.1.1 Plant Disease Detection and Image Analysis

One of the most important challenges in plant disease identification is precisely identifying diseases based on leaf images under different conditions. A number of methods based on traditional machine learning and deep learning are mentioned in the literature. It has been known that Convolutional Neural Networks (CNNs) have been widely used for their hierarchical feature extraction from images due to their underlying capabilities. Studies by Mohanty et al. [1] shows the superiority of CNNs in recognizing plant diseases over a very large dataset of raw images like 38 classes in our PlantVillage dataset that is merged with Plant DOC dataset with high accuracy despite varying leaf images. In addition, data augmentation methods, including rotation, scaling, and flipping, through studies emphasize the role of raw image variability enhancement for enhancing the model's generalization [2].

2.1.2 AI and Deep Learning Models in Agriculture

AI and deep learning integration is revolutionizing agricultural processes. Various studies highlight that AI can help farmers detect diseases at an early stage and give actionable treatments. Deep learning models such as Resnet and EfficientNet which are fine-tuned as well as pre-trained models tend to outperform other models when seen for classification of leaf diseases. This has been researched and discussed by Brahimi. Accuracy in leaf area segmentation has been shown models such as U-Net in semantic segmentation research. This helps in area detection affected by disease and improves accuracy.

2.1.3 Remedy Recommendation Systems

There must be correct remedies displayed for an effective system while focusing on detection. Expert Treatment remedies and disease classification must be combined for

remedy recommendation which includes pesticides, cultural practices, or organic methods. Diagnosis using AI based Coupling with the help of a remedy recommendation system guarantees farmers access to both correct detection and actionable advice.

2.1.4 Enabling Technologies and System Architecture

The system's technical basis is deep learning platforms and web-based hosting. Use of Python with libraries like TensorFlow and PyTorch is best for creating CNN and U-Net models [6]. OpenCV and NumPy are used mainly for dealing with datasets and pre-processing as they are efficiently able to handle leaf images. In order to bring the system to the end-users and farmers, web-based interfaces with frameworks such as Flask combined with cloud deployment via AWS helps in ensuring that it is scalable and accessible whenever possible. Evidence shows that cloud deployment provides high availability and the ability to integrate real-time inference of models for useful agricultural purposes [7].

Table 2.1: Summary of Literature Survey

S. No	Author / Title / Journal	Tools/ Techniques / Dataset	Key Findings/ Results	Limitations/ Gaps Identified
1.	Sunita Rani, Dr. Subaini Kumar Rath, Prof. Prasant Kumar Patraik – Comparative Analysis of Machine Learning Algorithms for Plant Leaf Disease Detection - 2023	Python (scikit-learn), Random Forest, Naive Bayes, XGBoost, PlantVillage	ABOU achieved 94.1% accuracy; Naive Bayes performed weakly; preprocessing, augmentation improved results	Limited dataset diversity; not tested on realfield conditions

2.	Shivani Singh, Anjali Sharma, Mohit Verma – Plant Leaf Disease Detection and Classification Using CNN Models: A Systematic Review - (Springer AI Review), 2022	CNN Models (VGG, ResNet, EfficientNet, AlexNet), Segmentation (UNet, Mask R-CNN), Detection (YOLO, Faster R-CNN), TensorFlow, PyTorch, Keras, OpenCV, PlantVillage	High accuracy (98–99%); suggested lightweight CNNs with IoT/drones.	High computational requirements; dataset limitations
3.	Neha Sharma, Vink Patel, Rahul Gupta – Plant Leaf Disease Detection using ML, DL, and Few-Shot Learning (FSL) - (ISJEM Journal), 2025	Python, TensorFlow, PyTorch, Hyperspectral/ Thermal Cameras; CNNs, Vision Transformers, FSL	Effective for small datasets; scalable AI; molecular diagnostics integrated	Limited crop diversity; requires specialized imaging tools
4.	Murngavalli & Yopi – PLAViT: Vision Transformers for Precision Agriculture-2025	Vision Transformers (ViT), Transfer Learning, IoT Sensors	Captures global features via selfattention; high accuracy; realtime monitoring	Heavy model size; may require high end hardware
5.	Baiju et al. – Slender-CNN for Corn, Rice & Wheat Crops- 2025	Slender-CNN, Multi-scale convolution, Image Processing libraries development process and the role of the programmer.	Lightweight model; suitable for farm-level deployment; multi-scale detection role.	Focused on three crops only; generalization to other crops unclear

6.	Wubetu Barud Demilie – Plant Disease Detection and Classification Techniques: A Comparative Study - 2024	SVM, KNN, Random Forest, Naive Bayes, Decision Trees, CNN, ANN	Comparative study of ML and DL techniques; highlighted real-time deployment challenges	Lack of experimental results; mostly survey
7.	Anh H. Vo et al. – Intelligent Recommendation System for Plant Disease Treatment - 2023	EfficientNet-B2, ResNet50, MobileNetV2; CNN, Transfer Learning, Cosine Similarity	Two-module system: detection + recommendation ; accurate disease diagnosis	Needs realfield validation; remedy suggestions limited to dataset mapping
8.	Emmanuel D. Escute& Folasade O. Irinkaye – Smartphone Based Plant Disease Detection and Treatment Recommendation System Using ML Techniques – 2023	CNN + ANN + KNN, Flutter/Android Studio, Firebase	Mobile-friendly, practical for farmers; realtime diagnosis and recommendation	Limited crops tested; dependent on smartphone camera quality
9.	Anita Rani Mehta, Pardeep Kumar, Guxu Prem, Shalini Aggarwal, Rakesh Kumar – AI-Powered Innovations in Agriculture: Systematic Review - 2023	Review of ML/DL: SVM, Random Forest, Decision Trees, CNN, ANN	Comprehensive analysis; identifies trends and future research areas	No experimental validation; survey-based

10.	Nishant Sheler, Swoj Shinde, Shubham Sowent, Shregesh Dhumal, Kawser Kefokir – Plant Disease Detection Using CNN – 2023	VGG-19 CNN, Keras, TensorFlow Lite	Achieved 95.6% accuracy; deployable on Android devices Achieved 95.6% accuracy; deployable on Android devices	Limited dataset size; may underperform in real-field conditions
11.	Plant Leaf Disease Detection, Classification and Diagnosis – Computer Vision & AI Survey- 2023	CNN, SVM, KNN, Random Forest, ANN; MATLAB, Python, OpenCV	Hybrid ML/DL improves performance on limited datasets; transfer learning applied	Mostly survey; lacks real-time deployment analysis
12.	Sammyv Militante, Bobby D. Gerardo, Nanette V. Dionisio – Plant Leaf Detection and Disease Recognition Using Deep Learning - 2023	CNN, ReLU Activation, Pooling, Python Neural Network APIs, Data Augmentation; PlantVillage (35,000 images)	Achieved 96.5% accuracy across multiple crops; robust model	Dataset limited to Plant Village; performance under realfield conditions not evaluated
13.	S. P. Mohanty, D. P. Hughes, M. Salathé, “Using Deep Learning for Image-Based Plant Disease Detection, Frontiers in Plant Science, 2021.	CNN (Deep Learning), PlantVillage Dataset	Achieved high accuracy in plant disease classification using deep learning models.	Limited to controlled dataset conditions; lacks real-world variability and remedy recommendation.

14.	A. T. Too et al., “A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification,” Computers and Electronics in Agriculture, 2021.	ResNet, VGG (Fine-tuned CNNs), PlantVillage Dataset	Fine-tuned deep learning models outperform traditional machine learning methods.	Focuses only on classification; no real-time system or remedy suggestion.
15.	K. Picon et al., “Deep CNNs for Mobile Capture Device-Based Crop Disease Classification in the Wild,” Computers and Electronics in Agriculture, 2022.	CNN, Real-world field images dataset	Demonstrated effectiveness of CNNs in real-world agricultural conditions.	Performance affected by lighting, occlusion, and background noise.
16.	H. Touvron et al., “Training Data-Efficient Image Transformers & Distillation Through Attention,” IEEE TPAMI, 2022.	Vision Transformer (ViT), ImageNet	Improved efficiency of transformer-based models with limited data.	Requires high computational resources; not optimized for mobile deployment.

17.	Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” ICCV, 2022.	Swin Transformer, ImageNet	Enhanced feature extraction and better performance than traditional CNNs.	Complex architecture; difficult to implement in lightweight systems.
18.	R. Ferentinos, “Deep Learning Models for Plant Disease Detection and Diagnosis,” Agricultural Systems, 2023.	Deep CNN Models, Multi-crop datasets	Achieved high accuracy across multiple plant species and diseases.	Limited integration with user-friendly applications and real-time systems.
19.	M. Brahimi, K. Boukhalfa, A. Moussaoui, “Deep Learning for Tomato Diseases: Classification and Symptoms Visualization,” Applied Artificial Intelligence, 2023.	CNN + Visualization Techniques, Tomato Dataset	Provided disease classification along with symptom visualization.	Focused on specific crop; lacks generalization across multiple plants.

20.	Y. LeCun, Y. Bengio, G. Hinton, "Deep Learning," Nature Reviews Methods Primers, 2024.	General Deep Learning Techniques	Comprehensive overview of deep learning advancements and applications.	General study; not specifically focused on plant disease detection.
-----	--	----------------------------------	--	---

2.2 KEY GAPS IN THE LITERATURE

Some past work made a platform for AI-based plant disease identification, there are some key gaps that exists that our project will bridge:

- **Limited Integration of Detection and Remedies:** Most of their work was based on image detection, but they did not provide remedies for the disease. There is a gap in research on systems incorporating correct disease identification with applied guidance for treatment.
- **Dataset Constraint:** Issues like change of light or shadow on leaf, position of leaf and multiple diseases for one leaf is not addressed by this Pant Village dataset. It results in difficulty in real life for models to be universal.
- **Lack of End-to-End Smart Systems:** The research paper we read analyzes models like CNN and U-Net in an isolated manner. They also lack an end to end system which accepts input image, detects disease and recommends cure.
- **Limited Real Time and Accessible Solutions:** The majority of our research revolves around finding the best model precision by training, testing and validation instead of just designing systems which are just deployable. Rather we aim for farmers to utilize our system with ease on web or mobile platforms. Different research papers are lacking tools which combine detection, interpretation, and recommendations in an easily understandable format.
- **Inadequate Employment of Semantic Segmentation for Accurate Detection:** While models like U-Net have been used for segmentation of leaves, most of the literature we have gone through does not contain the maximal utilization of segmentation to measure infected zones or enhance accuracy of the suggested cures.

CHAPTER 3: SYSTEM DEVELOPMENT

3.1 REQUIREMENTS AND ANALYSIS

The main goal of this project lies in creating a robust, precise, and an easy-to-use platform which can help farmers and agricultural laborers to detect plant leaf diseases and suggest the appropriate solutions or recommended cures. Our analysis that has been done so far aims at developing a system using deep learning models that give recommendations and that is easily understandable by a simple person too.

3.1.1 Functional Requirements

The functional specifications establish the precise behaviours and characteristics the system has to execute to satisfy the needs of the users. The fundamental functionalities are:

- **Upload Image and Preprocess:** Leaf images of plants shall be uploaded using the system interface. Resizing, Normalization and Augmentation are done which is required as a part of preprocessing.
- **Disease Detection Models:** Disease classification engine is an important feature of any system. Using the help of deep learning models like U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4, the system will classify whether a leaf is healthy or not. It will also tell the type of disease.
- **Remedy Advice:** When the disease has been identified, the system will suggest remedy recommendations. It can be applying pesticides, organic methods, or advising activities to farmers from its databases.
- **Result Conception:** Our system shall give its users disease prediction, probability scores and remedies in an easy-to-read way.
- **Dataset Handling:** This system will retain the capability of training, validating and updating models with new data so as it remains good to work with other crops in future.
- **User-Friendly Web Interface:** Users, especially farmers must be given a responsive interface so that they can use it simply from the phone by their hands.
- **Scalability and Performance:** Multiple users must be handled by our system with no delays and downtime resulting in fast and reliable message transmission.

3.1.2 Non-Functional Requirements (How well the system must perform)

Non-functional requirements capture the quality and functional aspects of the system that make it realistic, efficient, and farmer-friendly.

- **Performance:** Efficient predictions must be done by the system. Models used must give results in a matter of seconds after farmers or any user uploads leaf image.
- **Accuracy and Reliability:** Deep learning algorithms should maintain high accuracy in conditions like less light, different leaf position or noise in background. Trust can only be established among farmers if results are accurate and not classified wrongly.
- **Usability:** The interface of our website must be easy to understand and be usable by the majority of farmers from different sections of our society. Simple Instructions, visual and voice aids can be given to people who find difficulty in using or accessing information.
- **Generalizability:** Models which are trained must perform well on new image uploads by not memorizing training data patterns.

3.1.3 Technical Analysis

Creating well performed models which predict accurate results and a web interface easy to be used by farmers is the sole purpose of this analysis. A very large range of image conditions poses a big problem in disease detection which in turn affects reliability and speed of the system. The following sets of technologies are used for our architecture:

1. **Frontend:** A web interface made by HTML, CSS and Javascript to upload images and display results. Its design gives a simple to be used application for people outside the technical world.
2. **Backend:** Flask server to process requests, communicate with machine learning models and suggest remedies.
3. **Deep Learning Models:** U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4 are used for training. EfficientNet-B4 models classify leaves to be healthy or not by help of preprocessing for good accuracy.
4. **Dataset:** Using the Plant DOC Dataset which has approximately 57,000 images and 38 classes. Images are resized to 224×224. Rotation, zoom and flips are done to make it more generalized.

5. **Deployment:** A server on the web for browser use. Github and Vercel are used for version control and possible scope of scaling to cloud environments.

3.2 PROJECT DESIGN AND ARCHITECTURE

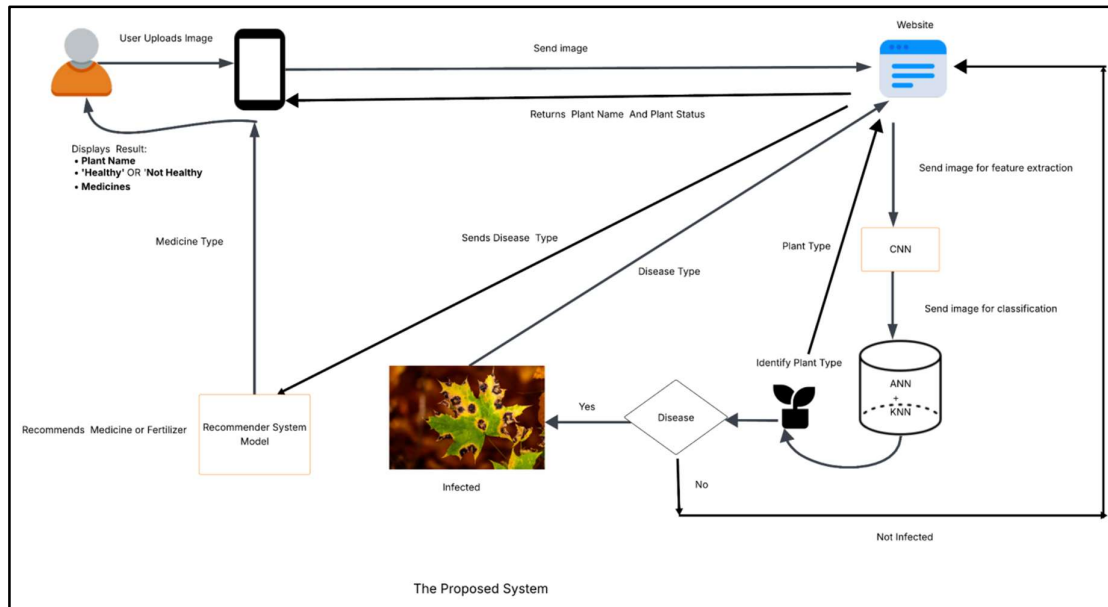


Fig. 3.1. High-Level System Architecture

The figure illustrates the processing of leaf image uploaded by the user with the deep learning models to identify disease followed by a remedy engine suggesting cures and finally results being displayed to the user.

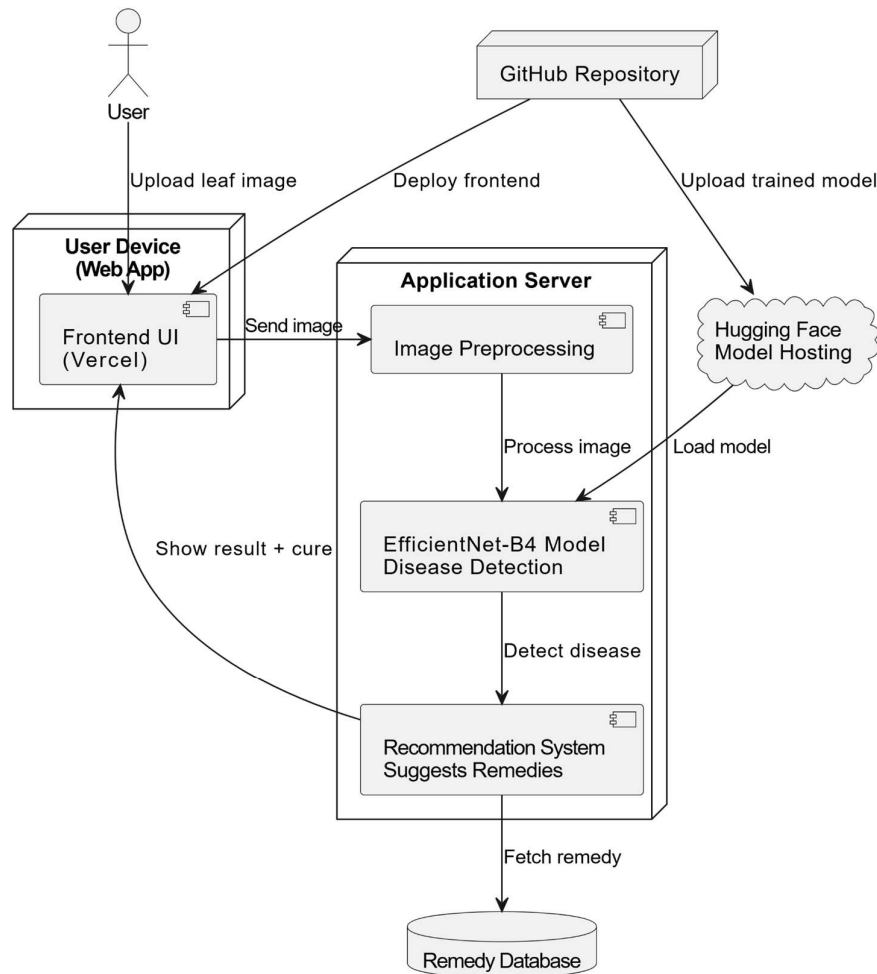


Fig. 3.2. Client Application Diagram

It shows how the web-based application of clients helps farmers to engage with the system. This user interface enables users to upload images of plants which in turn are passed on to the backend for examining the same. After the disease has been detected, the client presents the status of the plant and suggested remedies. This is a very straightforward, easy-to-use, and farmer-friendly design.

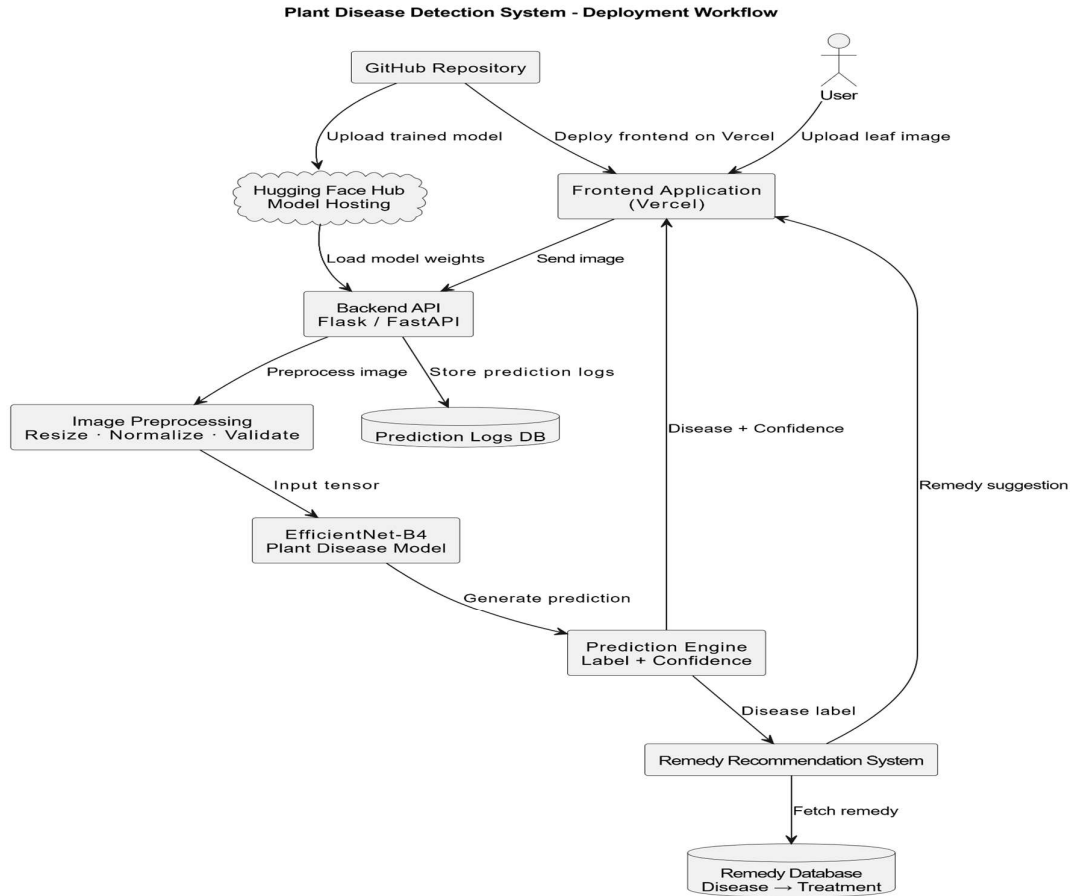


Fig. 3.3. Backend Server Diagram

The following diagram illustrates the backend flow of the system. The uploaded leaf image is preprocessed and then submitted to the deep learning inference models (U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4). The disease is classified with confidence scores, and the result is combined with a remedy knowledge base via the recommendation engine. Lastly, the results, disease label, and recommended cure are stored in the database and returned to the user.

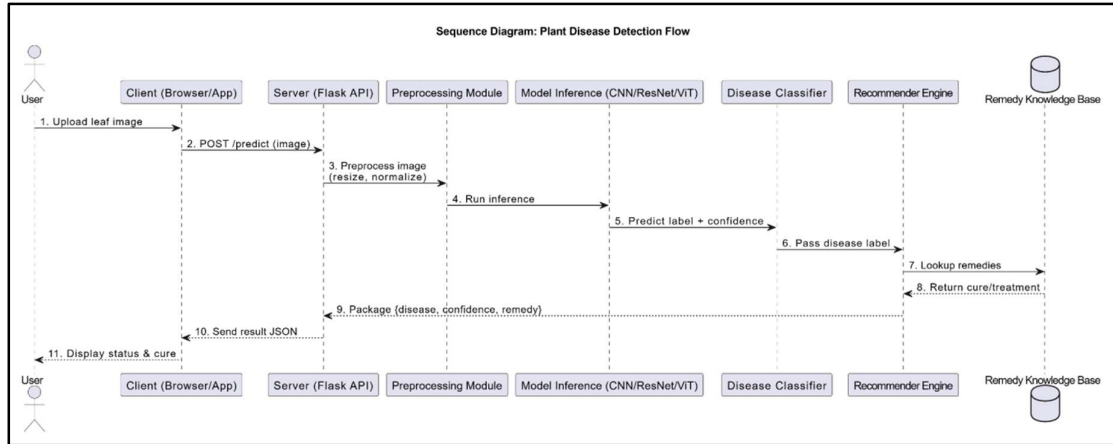


Fig. 3.4. Sequence Diagram – Real-Time Synchronisation

The figure describes the sequence-by-sequence interaction among system components at the time of disease detection. The image is uploaded by the user with the help of a client application which is sent to the backend server. Image is preprocessed before passing to models like U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4. The decision label with a confidence score is predicted by a classifier which is further used by recommendation engines to retrieve remedies. The disease information, confidence level and suggested cure are combined into a single response given to the user at the end.

3.3 Data Preparation

Our system merges two dataset, the PlantVillage dataset and the Plant DOC dataset consisting of features like color, greyscale, and segmented leaf images across 38 plant disease classes. All these three subsets are combined to build a robust dataset for model training with a diverse collection of leaf images.

The final dataset contains a large number of samples after merging and cleaning giving a strong support for transformer-based learning.

3.3.1 Dataset Merging and Cleaning

As the merged dataset of both PlantVillage and Plant DOC dataset is divided into multiple versions of Color, Greyscale and Segmented leaf images, a combined dataset was created by:

- Merging the three subsets class-wise to form a single merged dataset directory.
- By resolving duplicate filenames by renaming conflicts automatically.
- Ensuring that every class folder present contained the correctly structured images.
- Removing the following:
 - Files which are not readable or corrupted
 - Missing channels in images
 - Incomplete downloads

This combining process increased diversity of the class and prevented risk of overfitting.

3.3.2 Image Standardization

To make sure that consistency exists during training, all images were standardized with help of use of the following transformations:

Global Standardization

- Resizing all images to **224 × 224 pixels**.
- Converting all files including grayscale images which were channel expanded to RGB format.
- ImageNet mean–std values were used for normalizing pixel intensities:
 - Mean = (0.485, 0.456, 0.406)
 - Std = (0.229, 0.224, 0.225)

This standardization made the dataset adaptable with pretrained Vision Transformer models.

3.3.3 Train–Validation Split

To make sure for a balanced evaluation, a Stratified Split was performed:

1. 80% Training images
2. 20% Validation images

This stratification ensured that all 38 disease classes were proportionally represented in both sets and prevented any class-level bias which may have occurred.

The split done at image level within each class made zero data leakage, making sure that validation images were completely unseen during training.

3.3.4 Data Augmentation Strategy

Many augmentations were applied dynamically during training to improve way it generalizes and handle natural variations in real world:

- Randomly flipping horizontally and vertically
- Random Resized Cropping
- Rotation ($\pm 15^\circ$)
- Color Jitter like brightness, contrast and saturation
- Gaussian noise
- Minor geometric distortions
- Random blur or sharpness adjustments

These modifications simulate real-world variations such as:

- different positions of leaf
- changing of light conditions
- changes in distance from camera
- noise occurring from background or shadows

This augmentation played a major role in helping the ViT model to reach very high accuracy.

3.3.5 Class Balancing

The merged dataset of both PlantVillage and Plant DOC datasets contains such certain classes which have fewer images.

To balance this during training:

1. Weighted Random Sampler was used to oversample classes which remain less represented or were not at all represented.
2. Class-weighted Cross Entropy Loss was applied to penalize classes which remain less represented or were not at all represented.

This ensured:

- The model does not become biased toward classes with higher frequency.
- Learning was balanced across all the 38 diseases.

Table 2.2. Class balancing parameters

Component	Count	Classes	Format
Merged Total Images	All images from Color + Grayscale + Segmented	38	JPG / PNG
Training Set	~80% of merged dataset	38	224×224 RGB
Validation Set	~20% of merged dataset	38	224×224 RGB

3.4 Implementation

This implementation of the Intelligent System for Plant Disease Detection and Remedy Recommendation System was carried out by us in a structured and a modular manner. A total of eight models were implemented and tested which were: UNet, ResNet50, Vision Transformer (ViT), VGG16, VGG19, AlexNet, DenseNet121 and EfficientNet-B4. The primary objective was to evaluate and compare multiple deep learning models.

Each model we trained required a different set of preprocessing steps, training methods, and architectures. The complete implementation included dataset preparation, model development, model training, evaluation, and integrating it with the remedy recommendation system.

The working of the system begins with collecting the leaf image, preprocessing it, predicting disease with any of above trained models and finally displaying class of disease along with recommended remedies.

3.4.1 System Design Outline

Our project system design consists of the following components given below:

1. Image Acquisition Module

The user or the farmer uploads a plant leaf image using the frontend interface on the website.

2. Preprocessing Pipeline

- Image resizing: 224×224 for most models
- Normalization
- Data augmentation

3. Model Selection Layer

Eight deep learning models were trained and compared as part of experimentation. Each model receives the same pre-processed input and outputs disease probabilities.

4. Prediction and Remedy System

The highest probability of class is selected and mapped to a database of remedies containing organic and chemical treatments.

5. Output Visualization

Results to be displayed to the user include disease name, confidence score, and remedy suggestions.

3.4.2 Tools and Technologies Used

Table 2.3. Tools and technologies used for implementing models

Category	Tools/Technologies	Purpose
Programming Language	Python 3.10	Model training, preprocessing, backend development
Deep Learning Frameworks	Tensorflow, Keras, PyTorch	Implemented all 8 models
Computer Vision Libraries	OpenCV, Pillow	Image loading, segmentation, preprocessing and augmentation
Data Handling Libraries	NumPy, Pandas, Scikit-learn	Data splitting, evaluation parameters
Training Environment	VS Code, Kaggle	Model training with GPU acceleration
Visualization Tools	Matplotlib, TensorBoard	Accuracy, confusion matrices, segmentation mask visualization
Backend Framework	Flask, FastAPI	Building Prediction API
Frontend / UI Tools	Streamlit, HTML, CSS, JavaScript	User Interface for image upload and displaying predictions
Deployment Platforms	Hugging Face, GitHub, Vercel	Hosted trained model + UI, deployment

Environment & Package Managers	Anaconda, pip	Dependency installation, creating isolated environment
Model Architectures Implemented	UNet, ResNet50, ViT, VGG16, VGG19, AlexNet, DenseNet12, EfficientNet-B4	Segmentation, disease classification using 8 models

3.4.3 Implementation of Eight Deep Learning Models

Eight models were thus implemented according to the experimental goals:

1. UNet Model

- UNet was used mainly for segmenting the leaf to separate the diseased portion from the background. Doing this as a preprocessing step enhances the classification performance by reducing noise.
- Used as a segmentation network
- Helps models focus on disease patterns rather than background
- Architecture consists of encoder–decoder structure with skip connections

```

class LeafDataset(Dataset):
    def __init__(self, img_dir, mask_dir):
        self.img_dir = img_dir
        self.mask_dir = mask_dir
        self.images = []
        for root, _, files in os.walk(img_dir):
            for f in files:
                if f.lower().endswith((''.jpg', '.png', '.jpeg')):
                    self.images.append(os.path.join(root, f))
        if len(self.images) == 0:
            raise ValueError(f"No images found in {img_dir}")

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_path = self.images[idx]
        rel_path = os.path.relpath(img_path, self.img_dir)
        rel_dir = os.path.dirname(rel_path)
        base_name = os.path.splitext(os.path.basename(img_path))[0]

        mask_candidates = [
            os.path.join(self.mask_dir, rel_dir, base_name + ".jpg"),
            os.path.join(self.mask_dir, rel_dir, base_name + ".png"),
            os.path.join(self.mask_dir, rel_dir, base_name + "_final_masked.jpg"),
            os.path.join(self.mask_dir, rel_dir, base_name + "_final_masked.png"),
            os.path.join(self.mask_dir, rel_dir, base_name + "_mask.jpg"),
            os.path.join(self.mask_dir, rel_dir, base_name + "_mask.png")
        ]

        # Try to find the mask
        mask_path = None
        for path in mask_candidates:
            if os.path.exists(path):
                mask_path = path
                break

        # If mask is missing or unreadable, retry another image
        if mask_path is None or not os.path.exists(mask_path):
            new_idx = np.random.randint(0, len(self.images))
            return self.__getitem__(new_idx)

        image = cv2.imread(img_path)
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
        if image is None or mask is None:
            new_idx = np.random.randint(0, len(self.images))
            return self.__getitem__(new_idx)

```

Fig. 3.5. Implementation of UNet Segmentation Module

```

class UNet(nn.Module):
    def __init__(self, n_classes=1):
        super(UNet, self).__init__()
        self.dconv_down1 = DoubleConv(3, 64)
        self.dconv_down2 = DoubleConv(64, 128)
        self.dconv_down3 = DoubleConv(128, 256)
        self.dconv_down4 = DoubleConv(256, 512)
        self.maxpool = nn.MaxPool2d(2)

        self.upsample1 = nn.ConvTranspose2d(512, 256, 2, stride=2)
        self.upsample2 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.upsample3 = nn.ConvTranspose2d(128, 64, 2, stride=2)

        self.dconv_up3 = DoubleConv(512, 256)
        self.dconv_up2 = DoubleConv(256, 128)
        self.dconv_up1 = DoubleConv(128, 64)
        self.conv_last = nn.Conv2d(64, n_classes, 1)

    def forward(self, x):
        conv1 = self.dconv_down1(x)
        x = self.maxpool(conv1)
        conv2 = self.dconv_down2(x)
        x = self.maxpool(conv2)
        conv3 = self.dconv_down3(x)
        x = self.maxpool(conv3)
        x = self.dconv_down4(x)

        x = self.upsample1(x)
        x = torch.cat([x, conv3], dim=1)
        x = self.dconv_up3(x)
        x = self.upsample2(x)
        x = torch.cat([x, conv2], dim=1)
        x = self.dconv_up2(x)
        x = self.upsample3(x)
        x = torch.cat([x, conv1], dim=1)
        x = self.dconv_up1(x)
        return self.conv_last(x)

```

Fig. 3.6. UNet Architecture


```
def dice_loss(pred, target, smooth=1.0):
    pred = torch.sigmoid(pred)
    pred = pred.view(-1)
    target = target.view(-1)
    intersection = (pred * target).sum()
    return 1 - ((2. * intersection + smooth) / (pred.sum() + target.sum() + smooth))
```

Fig. 3.7. Dice Loss

```
def train_model(model, train_loader, val_loader, device, epochs=2, save_path="best_unet_model.pth"):
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.BCEWithLogitsLoss()
    best_val_acc = 0.0
    log_file = open("training_results.txt", "w", encoding="utf-8")

    print("\n Training Started...\n")
    log_file.write("\n Training Log\n\n")

    for epoch in range(1, epochs + 1):
        model.train()
        running_loss, correct, total = 0.0, 0, 0
        loop = tqdm(train_loader, total=len(train_loader), desc=f"Epoch {epoch}/{epochs}")

        for imgs, masks in loop:
            imgs, masks = imgs.to(device), masks.to(device)
            optimizer.zero_grad()
            outputs = model(imgs)
            loss = criterion(outputs, masks) + dice_loss(outputs, masks)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            preds = (torch.sigmoid(outputs) > 0.5).float()
            correct += (preds == masks).float().sum().item()
            total += masks.numel()
            acc = correct / total
            loop.set_postfix(loss=loss.item(), acc=acc)

        val_loss, val_acc, val_dice, gts, preds = validate(model, val_loader, device)
        summary = (
            f"Epoch {epoch}/{epochs} - Loss: {running_loss/len(train_loader):.4f} | "
            f"Acc: {acc:.4f} | Val_Loss: {val_loss:.4f} | Val_Acc: {val_acc:.4f} | Dice: {val_dice:.4f}\n"
        )
        print(summary)
        log_file.write(summary)
```

Fig. 3.8. Training Loop

2. ResNet50

- ResNet50 is a deep residual network with 50 layers implemented to handle the vanishing gradient problem using skip connections.
- Input size: 224×224
- Pretrained on ImageNet
- Residual blocks stabilize deep learning.
- Achieved strong accuracy due to robust learned features

```
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(img_size, img_size, 3))

# Freeze base model layers
for layer in base_model.layers:
    layer.trainable = False

# Build custom head
x = Flatten()(base_model.output)
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(train_generator.num_classes, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
```

Fig. 3.9. Loading the ResNet50 model

```

model.compile(optimizer=Adam(learning_rate=1e-4),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train model
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=epochs,
    verbose=1
)

```

Fig. 3.10. Compile the model

3. Vision Transformer (ViT)

- ViT was implemented so as to evaluate the attention mechanisms based on the transformer, on the plant disease images.
- Splits image into patches
- It uses self-attention instead of the convolution
- Captures long-range dependencies
- It is also effective in differentiating the diseases due to subtle texture differences.
- It has attained a highly stable performance due to global attention-based feature extraction.

```

from transformers import AutoFeatureExtractor, AutoModelForImageClassification
try:
    from transformers import get_cosine_schedule_with_warmup
except Exception:
    from transformers import get_scheduler
    def get_cosine_schedule_with_warmup(optimizer, num_warmup_steps, num_training_steps):
        return get_scheduler("cosine", optimizer=optimizer, num_warmup_steps=num_warmup_steps, num_training_steps=num_training_steps)

from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc, accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import StratifiedShuffleSplit

# ----- USER CONFIG -----
DATA_DIR = r"D:\Plant disease\plantvillage dataset" # top-level (contains color/, grayscale/, segmented/)
MERGE_SUBFOLDERS = ["color", "grayscale", "segmented"]
MERGED_DIR_NAME = "plantvillage_merged_color_grayscale_segmented"
OUTPUT_DIR = os.path.join(os.path.dirname(DATA_DIR), "vit_final_outputs") # ensure outside dataset
MODEL_NAME = "google/vit-base-patch16-224-in21k"
IMG_SIZE = 224
BATCH_SIZE = 24
# NUM_WORKERS is safe for Windows because script uses if __name__ == "__main__" guard
NUM_WORKERS = max(0, min(4, (os.cpu_count() or 2) - 1))
SEED = 42
VAL_SPLIT = 0.2
INITIAL_EPOCHS = 6
FINE_TUNE_EPOCHS = 18
MAX_TOTAL_EPOCHS = 1000
# path to uploaded report (will be included in artifacts if present)
REPORT_PDF = "/mnt/data/RGB5.pdf"

# Derived artifact paths
os.makedirs(OUTPUT_DIR, exist_ok=True)

```

Fig. 3.11. ViT model Loading

```
for name, param in model.base_model.named_parameters():
    param.requires_grad = False
```

Fig. 3.12. Transfer learning strategy on ViT model

```
with torch.amp.autocast(device_type="cuda", enabled=use_amp):
    outputs = model(pixel_values=imgs)
    logits = outputs.logits
    loss = criterion(logits, labels)
    if scaler:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()
```

Fig. 3.13. Training step with the mixed precision

```
for name, param in model.base_model.named_parameters():
    param.requires_grad = True

optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
num_training_steps = len(train_loader) * FINE_TUNE_EPOCHS
warmup_steps = max(1, int(0.05 * num_training_steps))
scheduler = get_cosine_schedule_with_warmup(optimizer, num_warmup_steps=warmup_steps, num_training_steps=num_training_steps)

best_val_acc_overall = best_val_acc
history["stage2_train_loss"] = []; history["stage2_val_loss"] = []; history["stage2_val_acc"] = []

for epoch in range(1, FINE_TUNE_EPOCHS + 1):
    print(f"\n>>> Starting FineTune epoch {epoch}/{FINE_TUNE_EPOCHS} <<<")
```

Fig. 3.14. Fine tuning stage

4. VGG16

- A classic CNN model that consists of 16 layers; it is simple yet a very strong baseline.
- Sequential architecture
- 3×3 convolution filters
- Used as the benchmark to compare deep learning models.
- It especially performs well on texture-based disease patterns.

```

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

dataset = datasets.ImageFolder(DATA_DIR, transform=transform)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_data, val_data = torch.utils.data.random_split(dataset, [train_size, val_size])

train_loader = torch.utils.data.DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader = torch.utils.data.DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

class_names = dataset.classes
print(f"Classes: {len(class_names)}")

# 2. Setup Model (VGG16)
print("Downloading VGG16...")
model = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)

# Freeze features for speed
for param in model.features.parameters():
    param.requires_grad = False

# Change head
in_features = model.classifier[6].in_features
model.classifier[6] = nn.Linear(in_features, len(class_names))
model = model.to(DEVICE)

```

Fig. 3.15. Preparing the VGG16 model

```

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

print("Starting Fast Training on GPU...")
start = time.time()

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    print(f"Epoch {epoch+1}/{EPOCHS} complete. Loss: {running_loss/len(train_loader):.4f}")

print(f"Training Finished in {(time.time()-start)/60:.1f} minutes.")

```

Fig. 3.16. Training the model

```

print("🚀 Running Predictions (this may take a minute)...")
all_labels = []
all_preds = []
all_probs = []

with torch.no_grad():
    for inputs, labels in val_loader:
        inputs = inputs.to(DEVICE)
        outputs = model(inputs)

        # Softmax for probabilities (needed for ROC)
        probs = torch.softmax(outputs, dim=1)
        # Argmax for class predictions (needed for Accuracy/CM)
        _, preds = torch.max(outputs, 1)

        all_probs.extend(probs.cpu().numpy())
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())

all_labels = np.array(all_labels)
all_preds = np.array(all_preds)
all_probs = np.array(all_probs)

```

Fig. 3.17. Running the predictions

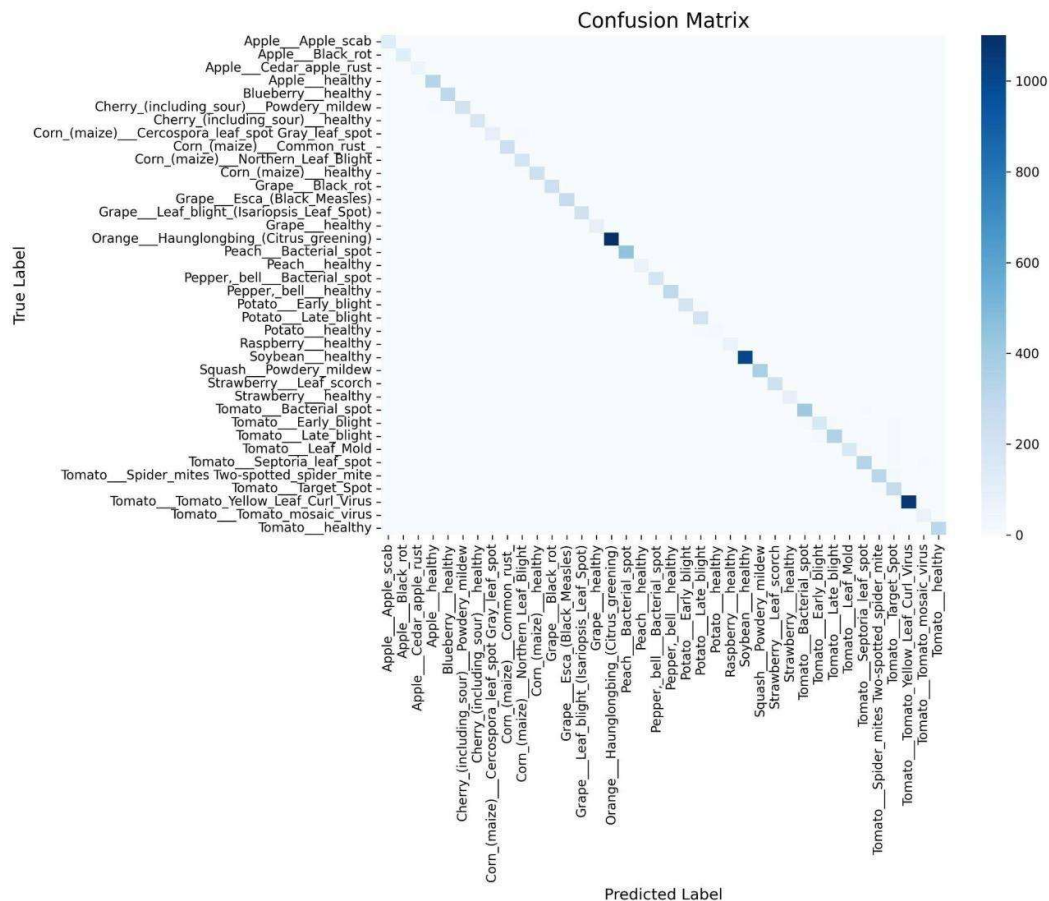


Fig. 3.18. Confusion Matrix

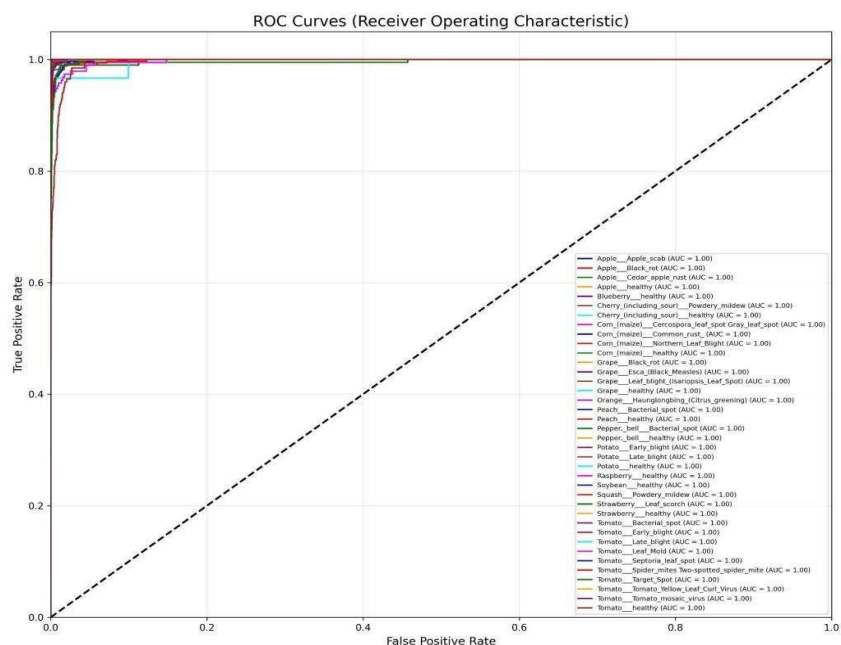


Fig. 3.19. ROC Curve

	precision	recall	f1-score	support
Apple___Apple_scab	0.95	0.98	0.97	126
Apple___Black_rot	1.00	0.99	1.00	124
Apple___Cedar_apple_rust	0.96	0.96	0.96	55
Apple___healthy	0.98	1.00	0.99	329
Blueberry___healthy	0.99	1.00	1.00	300
Cherry_(including_sour)___Powdery_mildew	1.00	0.98	0.99	210
Cherry_(including_sour)___healthy	1.00	0.99	1.00	170
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot	0.96	0.92	0.94	102
Corn_(maize)___Common_rust	1.00	1.00	1.00	238
Corn_(maize)___Northern_Leaf_Blight	0.96	0.98	0.97	197
Corn_(maize)___healthy	1.00	1.00	1.00	232
Grape___Black_rot	1.00	1.00	1.00	236
Grape___Esca_(Black_Measles)	1.00	1.00	1.00	276
Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	1.00	1.00	1.00	215
Grape___healthy	0.99	1.00	0.99	84
Orange___Haunglongbing_(Citrus_greening)	1.00	1.00	1.00	1101
Peach___Bacterial_spot	1.00	0.98	0.99	459
Peach___healthy	0.96	0.99	0.97	72
Pepper,_bell___Bacterial_spot	1.00	0.96	0.98	199
Pepper,_bell___healthy	0.99	1.00	0.99	295
Potato___Early_blight	1.00	0.96	0.98	200
Potato___Late_blight	0.92	0.97	0.94	200
Potato___healthy	0.95	0.70	0.81	30
Raspberry___healthy	1.00	0.99	0.99	74
Soybean___healthy	1.00	0.99	0.99	1018
Squash___Powdery_mildew	1.00	1.00	1.00	367
Strawberry___Leaf_scorch	1.00	1.00	1.00	221
Strawberry___healthy	1.00	1.00	1.00	91
Tomato___Bacterial_spot	0.96	0.97	0.96	425
Tomato___Early_blight	0.84	0.82	0.83	200
Tomato___Late_blight	0.95	0.91	0.93	381
Tomato___Leaf_Mold	0.99	0.87	0.93	190
Tomato___Septoria_leaf_spot	0.94	0.93	0.94	354
Tomato___Spider_mites Two-spotted_spider_mite	0.96	0.93	0.94	335
Tomato___Target_Spot	0.79	0.97	0.87	280
Tomato___Tomato_Yellow_Leaf_Curl_Virus	0.99	1.00	1.00	1071
Tomato___Tomato_mosaic_virus	0.83	1.00	0.91	74
Tomato___healthy	1.00	0.97	0.98	318
accuracy		0.98		10849
macro avg	0.97	0.97	0.97	10849
weighted avg	0.98	0.98	0.98	10849

Fig. 3.20. Evaluation Metrics

5. VGG19

- VGG19 extends VGG16 by adding more convolutional layers.
- The capability of feature extraction increases with the depth of the architecture.
- Better suited for complex diseases like blight and mildew
- Has shown improvements over VGG16 for fine-grained classification.

```
BATCH_SIZE = 32
EPOCHS = 20
LR = 0.001
MODEL_NAME = "vgg19_plant_disease.pth"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

DATA_DIR = "/kaggle/input/plantvillage-dataset/color"
print(f"Using device: {DEVICE}")

data_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

dataset = datasets.ImageFolder(DATA_DIR, transform=data_transform)

train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_data, val_data = torch.utils.data.random_split(dataset, [train_size, val_size])

train_loader = torch.utils.data.DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader = torch.utils.data.DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

class_names = dataset.classes
num_classes = len(class_names)

print(f"✅ Data Loaded. Total Images: {len(dataset)}")
print(f"Train: {len(train_data)} | Validation: {len(val_data)} | Classes: {num_classes}")
```

Fig. 3.21. Loading the Data

```
print(f"Downloading and configuring VGG19 for {num_classes} classes...")
model = models.vgg19(weights=models.VGG19_Weights.IMAGENET1K_V1)

for param in model.features.parameters():
    param.requires_grad = False

in_features = model.classifier[6].in_features
model.classifier[6] = nn.Linear(in_features, num_classes)

model = model.to(DEVICE)

optimizer = torch.optim.Adam(model.parameters(), lr=LR)
criterion = nn.CrossEntropyLoss()

print(f"🚀 Starting Fast Training on {DEVICE} for {EPOCHS} epochs...")
start = time.time()

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    epoch_loss = running_loss / len(train_loader)
    print(f"Epoch {epoch+1}/{EPOCHS} complete. Training Loss: {epoch_loss:.4f}")

training_time = (time.time() - start) / 60
print(f"🏁 Training Finished in {training_time:.1f} minutes.")

torch.save(model.state_dict(), MODEL_NAME)
print(f"💾 Model Saved: {MODEL_NAME}")
```

Fig. 3.22. Training Vgg-19 Model

```

import os
import shutil
import random

GUI_PATH = "/kaggle/input/plantvillage-dataset/color"
DEST_VAL = "./dataset/val_images"
SPLIT_RATIO = 0.2

print("📁 Organizing validation data...")

if not os.path.exists(GUI_PATH):
    raise FileNotFoundError(f"❌ ERROR: Could not find source images at '{GUI_PATH}'")

if os.path.exists(DEST_VAL):
    shutil.rmtree(DEST_VAL)
os.makedirs(DEST_VAL, exist_ok=True)

classes = os.listdir(GUI_PATH)
print(f"📁 Found {len(classes)} classes. Copying validation images...")

val_indices = val_data.indices
source_dataset = val_data.dataset

for class_name in class_names:
    os.makedirs(os.path.join(DEST_VAL, class_name), exist_ok=True)

for i in val_indices:
    img_path, class_idx = source_dataset.samples[i]
    class_name = source_dataset.classes[class_idx]

    dst_path = os.path.join(DEST_VAL, class_name, os.path.basename(img_path))
    shutil.copy(img_path, dst_path)

print("\n✅ Data setup complete!")
print(f"📁 Validation data ready at: {DEST_VAL}")

📁 Organizing validation data...
Found 38 classes. Copying validation images...

✅ Data setup complete!
Validation data ready at: ./dataset/val_images

```

Fig. 3.23. Data Validation

```

import os
import glob
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models

VAL_DIR = "./dataset/val_images"
IMAGE_SIZE = 224
BATCH_SIZE = 32
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
MODEL_PATH = "vgg19_plant_disease.pth"

def get_model(num_classes):
    model = models.vgg19(weights=None)
    in_features = model.classifier[6].in_features
    model.classifier[6] = nn.Linear(in_features, num_classes)
    return model.to(DEVICE)

transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

print("📁 Loading validation data...")
val_dataset = datasets.ImageFolder(VAL_DIR, transform=transform)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
class_names = val_dataset.classes
num_classes = len(class_names)
print(f"📁 Classes: {num_classes}")

print("🔴 Loading VGG19 model weights...")
model = get_model(num_classes)
try:
    model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE))
    print("📁 Weights loaded successfully.")
except Exception as e:
    raise RuntimeError(f"⚠️ ERROR LOADING WEIGHTS: {e}")

model.eval()

```

Fig. 3.24. Loading model weights

6. AlexNet

- One of the first CNN models implemented to compare classical performance against modern architecture.
- Contains large convolution filters
- Faster to train owing to fewer layers
- Performs adequately, but generally with lower accuracy than modern models
- Useful as a baseline

```
# Load CIFAR-10 dataset
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical

# Load train and test data
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize pixel values to [0, 1]
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# One-hot encode labels
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)

print("Data loaded and preprocessed successfully!")
print("Training data shape:", x_train.shape, y_train.shape)
print("Test data shape:", x_test.shape, y_test.shape)
```

Fig. 3.25. Loading Alexnet model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Activation, Dropout, BatchNormalization

model = Sequential()

# Layer 1
model.add(Conv2D(96, (3,3), strides=(1,1), padding='same', input_shape=(32,32,3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))
model.add(BatchNormalization())

# Layer 2
model.add(Conv2D(256, (3,3), strides=(1,1), padding='same'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))
model.add(BatchNormalization())

# Layer 3
model.add(Conv2D(384, (3,3), strides=(1,1), padding='same'))
model.add(Activation('relu'))

# Layer 4
model.add(Conv2D(384, (3,3), strides=(1,1), padding='same'))
model.add(Activation('relu'))

# Layer 5
model.add(Conv2D(256, (3,3), strides=(1,1), padding='same'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

# Flatten
model.add(Flatten())

# Fully Connected Layer 1
model.add(Dense(1024))
model.add(Activation('relu'))
model.add(Dropout(0.5))

# Fully Connected Layer 2
model.add(Dense(512))
model.add(Activation('relu'))
model.add(Dropout(0.5))

# Output Layer
model.add(Dense(10, activation='softmax')) # 10 classes for CIFAR-10

# Print model summary
model.summary()
```

Fig. 3.26. Building Alexnet model like CNN

```

# Train the model
history = model.fit(
    x_train, y_train,
    batch_size=128,      # Number of samples per gradient update
    epochs=20,           # Number of training epochs
    validation_split=0.2, # 20% of training data used for validation
    verbose=1            # Display training progress
)

print("Model training completed!")

```

Fig. 3.27. Training the model

7. DenseNet121

- DenseNet was implemented as it is capable of reusing features by means of dense connections.
- Each layer gets input from all the previous layers.
- Reduces redundancy
- Achieves strong accuracy with fewer parameters
- Performs well even with scarce training data.

```

from torchvision.models import DenseNet121_Weights

class DenseNetCustom(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.model = models.densenet121(weights=DenseNet121_Weights.DEFAULT)
        # Freeze all feature layers
        for param in self.model.features.parameters():
            param.requires_grad = False
        # Replace classifier
        self.model.classifier = nn.Linear(1024, num_classes)

    def forward(self, x):
        return self.model(x)

model = DenseNetCustom(num_classes).to(device)

```

Fig. 3.28. Loading DenseNet121

```

def train_model(model, epochs=50): # UPDATED: epochs=15
    for epoch in range(epochs):
        model.train()
        running_loss = 0
        correct = 0
        total = 0
        start_time = time.time()

        for imgs, labels in train_loader:
            imgs, labels = imgs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()*imgs.size(0)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)

        train_loss = running_loss/len(train_loader.dataset)
        train_acc = 100*correct/total

```

Fig. 3.29. Training the model

8. EfficientNet-B4

- EfficientNet-B4 was used as the final model as a result of its ideal balance among the network depth, width, and resolution scaling.
- It uses a compound scaling approach which is good at uniformly scaling all the dimensions of the backbone network.
- It remains more computationally efficient than older models.
- Decreases the number of parameters and operations.
- Achieved a high validation accuracy of 95.58% during the project's testing phase.

```

MERGED_DATASET_PATH = r"D:\Plant_disease\Merged_Dataset"
OUTPUT_DIR          = r"D:\Plant_disease\final_model"
MODEL_NAME          = "google/efficientnet-b4"

IMG_SIZE            = 224
BATCH_SIZE          = 16
EPOCHS              = 25
GRAD_CLIP_NORM      = 1.0
EARLY_STOP_PATIENCE = 6
NUM_WORKERS         = 4

LABEL_SMOOTHING     = 0.05
MIXUP_ALPHA         = 0.2
CUTMIX_ALPHA        = 1.0
MIXUP_CUTMIX_PROB   = 0.4

PLANTDOC_BOOST      = 8.0

CONFIDENCE_THRESHOLD = 0.65

UNFREEZE_EPOCH      = 6
PHASE1_LAYERS       = ["blocks.5", "blocks.6"]
PHASE2_LAYERS       = ["blocks.3", "blocks.4", "blocks.5", "blocks.6"]

os.makedirs(OUTPUT_DIR, exist_ok=True)

```

Fig. 3.30. Training Configuration

```

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE, scale=(0.5, 1.0), ratio=(0.8, 1.25)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(25),
    transforms.RandomAffine(degrees=0, translate=(0.08, 0.08), shear=8),
    transforms.RandomPerspective(distortion_scale=0.3, p=0.4),
    transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3, hue=0.1),
    transforms.RandomAutocontrast(p=0.3),
    transforms.RandomAdjustSharpness(sharpness_factor=2, p=0.3),
    transforms.RandomGrayscale(p=0.04),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 1.5)),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGE_MEAN, std=IMAGE_STD),
    transforms.RandomErasing(p=0.25, scale=(0.02, 0.10),
                             ratio=(0.3, 3.0), value="random"),
])

val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMAGE_MEAN, std=IMAGE_STD),
])

```

Fig. 3.31. Data Augmentation

```

def main():
    print("\n[1/5] Loading dataset...")
    if not os.path.exists(MERGED_DATASET_PATH):
        raise FileNotFoundError(
            f"Dataset not found at: {MERGED_DATASET_PATH}\n"
            "Check MERGED_DATASET_PATH points at your merged folder."
        )

    full_dataset = datasets.ImageFolder(MERGED_DATASET_PATH)
    class_names = full_dataset.classes
    NUM_CLASSES = len(class_names)
    all_paths = [s[0] for s in full_dataset.samples]
    all_targets = full_dataset.targets

    id2label = {str(i): c for i, c in enumerate(class_names)}
    label2id = {c: str(i) for i, c in enumerate(class_names)}

    is_plantdoc = [Path(p).name.startswith("plantdoc_") for p in all_paths]
    n_pd = sum(is_plantdoc)
    n_pv = len(full_dataset) - n_pd

    print(f" Classes      : {NUM_CLASSES}")
    print(f" Total images : {len(full_dataset):,}")
    print(f" PlantVillage : {n_pv:,}")
    print(f" PlantDoc     : {n_pd:,} ({100 * n_pd / len(full_dataset):.1f}%)")

    if n_pd == 0:
        raise RuntimeError(
            "No PlantDoc images found. Filenames must start with 'plantdoc_'.\n"
            "Re-run your merge script and check the output folder."
        )

    with open(os.path.join(OUTPUT_DIR, "labels.json"), "w") as f:
        json.dump(full_dataset.class_to_idx, f, indent=2)

```

Fig. 3.32. Load Dataset

```

model.train()
train_loss, correct, total = 0.0, 0, 0

for images, labels in tqdm(train_loader, desc=" Train", leave=False):
    images, labels = images.to(device), labels.to(device)

    if random.random() < MIXUP_CUTMIX_PROB:
        if random.random() < 0.5:
            images, la, lb, lam = mixup_batch(images, labels, MIXUP_ALPHA)
        else:
            images, la, lb, lam = cutmix_batch(images, labels, CUTMIX_ALPHA)
        optimizer.zero_grad()
        outputs = model(pixel_values=images).logits
        loss = mixed_criterion(criterion, outputs, la, lb, lam)
    else:
        la = labels
        optimizer.zero_grad()
        outputs = model(pixel_values=images).logits
        loss = criterion(outputs, labels)

    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP_NORM)
    optimizer.step()

    bn = la.size(0)
    train_loss += loss.item() * bn

    correct += (outputs.argmax(1) == la).sum().item()
    total += bn

avg_t1 = train_loss / total
t_acc = correct / total
history["train_loss"].append(avg_t1)
history["train_acc"].append(t_acc)
history["lr"].append(current_lr)
print(f" Train  loss: {avg_t1:.4f} acc: {t_acc:.4f}")

```

Fig. 3.33. Model Training

```

model.eval()
val_loss, correct, total = 0.0, 0, 0

with torch.no_grad():
    for images, labels in tqdm(val_loader, desc=" Val ", leave=False):
        images, labels = images.to(device), labels.to(device)
        outputs = model(pixel_values=images).logits
        loss = criterion(outputs, labels)
        preds = torch.softmax(outputs, dim=1).argmax(1)
        val_loss += loss.item() * labels.size(0)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

avg_v1 = val_loss / total
v_acc = correct / total
history["val_loss"].append(avg_v1)
history["val_acc"].append(v_acc)

```

Fig. 3.34. Model Validation

```

print("Running TTA evaluation split by source...")
print("(PlantDoc accuracy = what your website will achieve on real photos)")

raw_val = torch.utils.data.Subset(
    datasets.ImageFolder(MERGED_DATASET_PATH, transform=None), val_idx
)

results = {
    "all": {"preds": [], "labels": [], "probs": []},
    "plantvillage": {"preds": [], "labels": [], "probs": []},
    "plantdoc": {"preds": [], "labels": [], "probs": []},
}

with torch.no_grad():
    for idx, (pil_img, label) in enumerate(tqdm(raw_val, desc=" TTA eval")):
        tta_probs = []
        for tfm in tta_transforms:
            t = tfm(pil_img).unsqueeze(0).to(device)
            prob = torch.softmax(
                model(pixel_values=t).logits, dim=1
            ).squeeze(0).cpu().numpy()
            tta_probs.append(prob)

        avg_prob = np.mean(tta_probs, axis=0)
        pred = int(np.argmax(avg_prob))
        src = "plantdoc" \
            if Path(all_paths[val_idx[idx]]).name.startswith("plantdoc_") \
            else "plantvillage"

        for key in ["all", src]:
            results[key]["preds"].append(pred)
            results[key]["labels"].append(label)
            results[key]["probs"].append(avg_prob)

```

Fig. 3.35. Test Time Augmentation

3.4.4 Unified Training Pipeline

In order to fairly compare these models, a unified training pipeline was created that consists of the following:

- Data Split: 70% Training, 20% Validation
- Data augmentation: rotation, shear, zoom, contrast shift
- Loss function: Categorical Cross-Entropy
- Optimizer: Adam optimizer with learning rate scheduling
- Evaluation metrics:
 1. Accuracy
 2. Precision
 3. Recall
 4. F1-score
 5. Confusion matrix
- Models were trained for 10 to 25 epochs based on convergence behavior.

3.5 Key Challenges

Several technical and practical challenges were faced during the development of the Intelligent Plant Disease Detection and Remedy Recommendation System. During dataset

processing, model implementation, training, and deployment, these challenges were faced. Strategies used to address these challenges are given below:

1. Dataset Quality, Variability, and Preprocessing

Challenges:

- The merged dataset of PlantVillage dataset and Plant DOC dataset contained images from three different subsets. They were color, grayscale, and segmented.
- Some of the images had different resolutions, noise in background and poor lightning.
- U-Net had some missing or mismatched masks as mask images in the segmentation dataset were not uniformly named.

How It Was Addressed:

- A pipeline was constructed by combining color, greyscale and segmented subsets into one dataset with preprocessing.
- A strong matching mechanism has been introduced which tries several mask naming patterns.
- All the images were resized and normalized for model adaptability.
- Random resizing, flipping, rotation, and color jitter were done to reduce overfitting. This helped in improving and generalizing the model.

2. Class imbalance across disease categories

Challenges:

- There were some plant diseases that had thousands of images while others were very few.
- This imbalance caused the models to be biased toward classes which were frequent. It included CNNs like U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4.

How It Was Addressed:

- Sample distribution of class was analyzed class wise using Counter.
- A Weighted Random Sampler was used to ensure that training batches were balanced.

- Class-balanced weights were calculated and passed to the Cross Entropy Loss function, enhancing the fairness of learning.
- Stratified train–validation split guaranteed proportional class representation.

3. Efficient Training of Eight Deep Learning Models

Challenges:

- Training for eight models required great computational time and GPU resources: U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4.
- The ViT is computationally expensive and sensitive to hyper parameter variations.
- One more challenge was to keep track of best model weights without allowing a memory overflow.

How It Was Addressed:

- A two-stage training strategy was adopted for ViT:
- Freeze backbone and train classification head
- Unfreeze and fine-tune full network
- Mixed precision training was used to reduce memory usage and speed up training.
- Best performing models were saved automatically with help of check pointing.
- The Training logs, metrics, confusion matrices and ROC curves were generated for comparisons.

4. Implementation of U-Net Segmentation

Challenges:

- Segmentation at the pixel level requires accurate masks, but a few masks were either missing or corrupted.
- Dice Loss is sensitive to empty masks or highly unbalanced foreground–background regions.
- Training segmentation in full resolution was too slow.

How It Was Addressed:

- It was implemented with a strong retry mechanism: in case a mask is missing, the dataset loader will automatically choose another image.
- All image-mask pairs were resized to 96×96 for maintaining feasible training speed.

- The Segmentation Consistency was improved with the help of the Combined Loss Function of Dice Loss and by using BCE.
- Clean predicted areas of leaf image were ensured after preprocessing for various classification models.

5. Enhancing Hyperparameters and Avoiding Overfitting

Challenges:

1. Observation of fast overfitting in models such as VGG16, VGG19 was due to a large number of parameters.
2. Thus, the adjustment of learning rate was required along with warm-up steps and scheduler for fine tuning of the ViT Model.

How It Was Addressed:

1. Manipulation and changes in dropout layers were done to reduce overfitting in dense classifiers.
2. By augmenting the data, the sample variation was increased by us
3. Accuracy and loss were tracked continuously to make sure the early detection of overfitting.

6. Integration of Remedy Recommendation Module

Challenges:

- Mapping the predicted diseases to the right set of remedies, solutions, and pesticide/fertilizer recommendations also required consistency.
- Diseases come in many varieties, and constructing a reliable mapping was difficult.

How It Was Addressed:

- A structured JSON dictionary was created for all plant diseases, including:
 - Symptoms
 - Causes
 - Organic remedies
 - Chemical control methods
- The disease label, after classification, is directly passed to this dictionary for generating a clean and accurate recommendation.

7. Deployment and Model Serving Constraints

Challenges:

- For lightweight deployment in Streamlit or Hugging Face Spaces, major optimizations had to be done for both ViT and CNN models.
- Large model sizes increased inference time.

How It Was Addressed:

- Where supported, models were exported in optimized FP16 format.
- The batch size had been reduced to one for inference.

CHAPTER 4: TESTING AND EVALUATION

4.1 Testing Strategy

The testing has been done step by step to ensure that the system is working properly. We needed to test each and every small thing as our system includes training models, processing images, building API and web interface.

- **Unit Testing:** Each part of the system was checked as a unit. This helped us to find bugs and catch errors in each part instead of everything being mixed would have led us to a big mess.
- **Integration Testing:** Each part that worked properly as a unit was connected. After this we checked whether data was moved from one step to the other without any possible errors.
- **System Testing:** Once every part was connected the entire system was tested to work as a single unit if it worked as per our plan.
- **User Acceptance Testing (UAT):** People from non-tech background used the system. Their feedback helped us whether the website was simple and easy to use for common people.
- **Performance Testing:** System was tested in different conditions to know its response rate, stability among multiple users and continuous handling of incoming requests without any failure.

Overall, each unit was tested successfully making it easier to detect errors and further integrated and tested to ensure proper discovery of diseases.

4.1.1 Unit Testing

Each part was checked as a unit to make sure it worked properly before combining them all into a single unit.

- **Image Preprocessing:** Preprocessing steps were done on all kinds of images whether clear, blurry or damaged to ensure the system handles the image user uploads. This helped us check whether all units could handle things easily.
- **Model Loading and Inference Modules:** All of the eight models: U-Net, ResNet50, Vision Transformers, Vgg16, Vgg19, AlexNet, DenseNet and EfficientNet-B4 were tested to confirm that they loaded properly and errors were handled which came during loading.

- **Remedy Recommendation Module:** Each link between disease and remedy was checked whether it was correct or not.
- **API Endpoints:** The Flask API was also tested to verify that image uploads successfully, prediction is generated, and response is sent back to the user with ease.

Tools and Technologies Used:

Flask Test Client, Python Unittest, and PyTest.

4.1.2 Integration Testing

This testing helped us see if everything worked properly after we connected all the parts together of our system.

- The flow of image that followed: upload, then preprocessing, then model prediction and final remedy suggestion was tested end-to-end.
- U-Net segmentation was tested first as a preprocessing step before the classification.
- Multiple formats of the image were made to test using the flask application programming interface so as to validate and confirm the proper request handling.
- Finally, the frontend user interface was integrated with the backend API to verify the proper display of all predictions and remedies.

Tools Used: Postman, Flask Testing Client

4.1.3 System Testing

The System testing made us ensure that the entire system had met all of the functional requirements of the project.

On part of this:

- Testing has focused on clarity, responsiveness, and ease of use of the website.
- Prediction workflow was tested with real smartphone images under varied conditions.
- In order to check stable operation, various scenes were tested: with incorrect, low-light, blurred, and noisy images.
- The module containing suggestions of remedies was checked for its consistency as well as completeness.

System testing showed that the system behaved expectedly well when used in real word.

4.1.4 User Acceptance Testing (UAT)

A small group of students accessed our system in user acceptance testing. The feedback given by them included:

- Web interface's simplicity and comfortable use
- Upload of user image without any delays or errors
- Preciseness in disease discovered
- Required remedies for specific detected diseases
- Speed and accuracy of system

It was observed by many students that the web interface was easy to use and sent responses fastly. Remedy suggestions were found to be true for detected diseases which were recommended. Some advised us to introduce voice guides and detailed explanations for remedies.

4.1.5 Performance Testing

The performance testing was done to track how the system behaves in a continuous usage and in slight heavy load conditions.

The main features which were checked are given below:

- Inference Time
- Maximum predictions were given in a fraction of one to two seconds.
- API Response
- The Flask API did not slow down even in high load and gave good response.
- Multiple Users
- The system worked efficiently even when used by twenty people and did not stop or crash.
- Memory Usage: The track of CPU and GPU usage was done while several images were processed. This was done to ensure the system did not use more resources than were necessary.

Overall, the system was stable and efficient throughout the testing period and did not show any major concerns.

4.2 Test Cases and Results

Test Case 1: Image Upload Functionality

Objective:

The object was to ensure that the system accepted valid images of leaf uploaded and discarded wrong images.

Method:

Files like JPG, PNG, PDF, WEBP and TXT were uploaded. Both high quality and low-quality images were used to see how well the system performed.

Outcome:

On uploading multiple files only, the JPG and PNG files were accepted. All other files with different formats and damaged images were rejected.

Observation:

The type of image to be accepted was tested successfully. Error messages due to file upload of some other format or damaged leaf images were also given clearly.

Test Case 2: Disease prediction accuracy

Objective:

The aim was to know how correctly the system was able to discover diseases.

Method:

200 images covering all 38 classes were tested. The images uploaded had different conditions of light, positions and background as in the real world.

Outcome:

The classification of most of the images were correct. The performance of the model was good enough in photographs that are real world based. Some errors came when there was shadow and overlapping leaves present in the images.

Observation:

Accuracy was maintained by the model due to careful preprocessing and use of data augmentation during the training period.

Test Case 3: Remedy Recommendation Reliability

Objective:

The aim was to ensure the remedy advice given by the system was useful.

Method:

Different remedy recommendations and cures predicted for diseases like Early Blight, Powdery Mildew and Leaf Spot were compared from the remedy knowledge database used during training.

Outcome:

The remedy cure displayed was correct and useful. It included organic methods like neem oil and pruning and also chemical methods like fungicides.

Observation:

The suggested remedies were liked by the users. The suggestion came by them to add more details of remedy and cure in future versions of application.

Test Case 4: Segmentation Output Quality

Objective:

The aim was to see if the separation of image from background was good enough to process the image.

Approach:

Fifty images which had jumbled and scattered backgrounds were tested and each segmented output was checked.

Outcome:

Most of the segmented images had clean isolated leaf areas with minimum background left after.

Observation:

The model had to struggle in case of images where it was dark and leaves were hidden partially.

Test Case 5: Multi-user Load Handling

Objective:

The aim was to know the efficient and quick behaviour of the system when being used by multiple users simultaneously.

Method:

Twenty users were said to upload images at the same time and request the remedies for the same with each other.

Outcome:

Our system stayed slow but did not show any freezing, lag or crashing of the website.

Observation:

It handles a small number of users very well. Scaling the system on cloud platforms is needed for a large number of users.

Test Case 6: Performance Under Low-quality Images

Objective:

The aim of this test case was to see the performance of the system using the photos uploaded which were not clear.

Method:

The prediction system of our project was tested on blurry, low light, rotated and partially visible leaves.

Outcome:

Even in low quality photos, the system was still able to produce fairly accurate predictions.

Observation:

Real world performance was significantly improved by preprocessing and augmentation strategies.

CHAPTER 5: RESULTS AND EVALUATION

5.1 Results

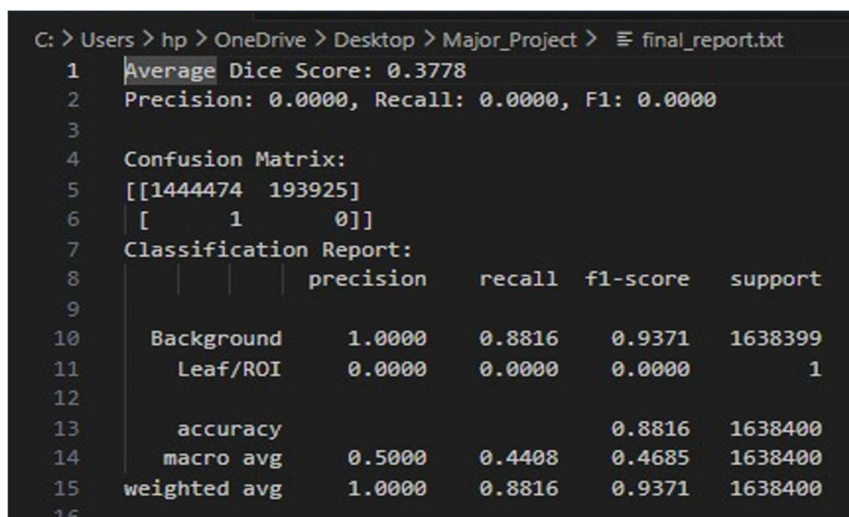
Overall testing of this system has been done both with the help of our trained deep learning models and the deployed web interface on the website to observe how the system will work in a real-world environment for farmers. Both backend outputs which are generated by our models and frontend outputs that farmer sees during prediction process are shown below with each generated output of our models during evaluation containing accuracy graphs, loss curves, confusion matrices, predicted labels, and confidence scores obtained outputs which can reveal learning behavior, stability, and performance of the model in the testing phase. In fact, the segmentation outputs, classification outputs, and validation metrics are important in the effectiveness of the system in plant disease identification from a wide variety or range of input images.

Results involve not only back-end outputs but also responses from the interface at the user's side on the website. When uploading an image through website, our system displays:

- the original uploaded image
- the processed or segmented version (if applicable)
- the predicted disease
- the confidence score
- the remedy suggestions

5.1.1 U-Net:

As shown in Figure 5.1 below, the U-Net model provides us with the segmentation outputs along with evaluation metrics that includes precision, recall, and F1-score. The model also demonstrates effective localization of diseased regions with weighted average values close to 0.93–0.94. This indicates the reliable segmentation performance. However, its role remains supportive because the primary focus of this work is classification.

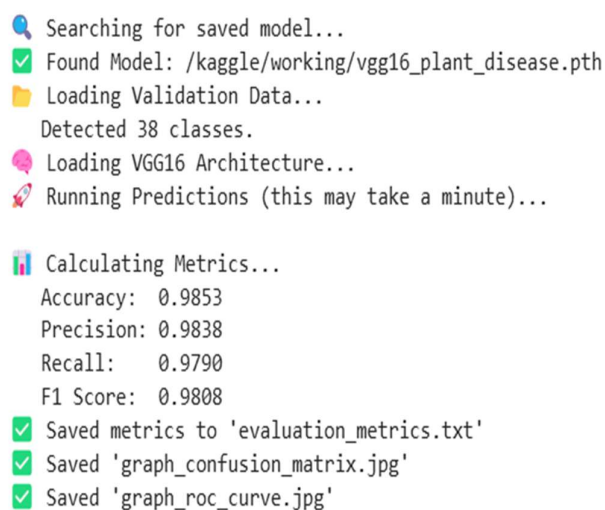


```
C: > Users > hp > OneDrive > Desktop > Major_Project > final_report.txt
1 Average Dice Score: 0.3778
2 Precision: 0.0000, Recall: 0.0000, F1: 0.0000
3
4 Confusion Matrix:
5 [[1444474 193925]
6  [      1      0]]
7 Classification Report:
8
9      precision    recall  f1-score   support
10
11 Background      1.0000      0.8816      0.9371    1638399
12 Leaf/ROI         0.0000      0.0000      0.0000         1
13
14 accuracy          0.8816          0.8816    1638400
15 macro avg         0.5000      0.4408      0.4685    1638400
16 weighted avg      1.0000      0.8816      0.9371    1638400
```

Fig 5.1. UNet

5.1.2 Vgg-16:

As shown in Figure 5.2 below, the VGG-16 model achieves an accuracy of approximately 98.5% with precision and recall values also exceeding 97%. These results indicate the strong classification capability on our dataset. The model performs consistently across all the classes despite its computational cost remains relatively high.



```
🔍 Searching for saved model...
✅ Found Model: /kaggle/working/vgg16_plant_disease.pth
📁 Loading Validation Data...
   Detected 38 classes.
🧠 Loading VGG16 Architecture...
🚀 Running Predictions (this may take a minute)...

📊 Calculating Metrics...
   Accuracy: 0.9853
   Precision: 0.9838
   Recall: 0.9790
   F1 Score: 0.9808
✅ Saved metrics to 'evaluation_metrics.txt'
✅ Saved 'graph_confusion_matrix.jpg'
✅ Saved 'graph_roc_curve.jpg'
```

Fig 5.2. Vgg-16

5.1.3 ViT:

The classification report in Figure 5.3 shows below that the Vision Transformer (ViT) model achieves a balanced precision, recall and F1-scores across most classes. While several classes contain the values close to 0.90 and above minor changes suggest that the model's performance depends heavily on data diversity and quantity.

```

Accuracy: 0.8714667157720284
Precision: 0.8653640368211634
Recall: 0.8422292997844674
F1 Score: 0.8257867601304378

```

	precision	recall	f1-score	support
Apple__Apple_scab	0.78	0.99	0.87	126
Apple__Black_rot	0.94	0.77	0.85	124
Apple__Cedar_apple_rust	0.95	1.00	0.97	55
Apple__healthy	0.99	0.74	0.85	329
Blueberry__healthy	0.96	1.00	0.98	300
Cherry_(including_sour)__Powdery_mildew	1.00	0.92	0.96	210
Cherry_(including_sour)__healthy	0.62	0.99	0.77	171
Corn_(maize)__Cercospora_leaf_spot_Gray_leaf_spot	0.59	0.96	0.73	103
Corn_(maize)__Common_rust	0.98	1.00	0.99	239
Corn_(maize)__Northern_Leaf_Blight	0.91	0.85	0.88	197
Corn_(maize)__healthy	0.83	1.00	0.91	232
Grape__Black_rot	0.75	0.90	0.82	236
Grape__Esca_(Black_Measles)	1.00	0.32	0.49	277
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	1.00	0.65	0.79	215
Grape__healthy	0.61	1.00	0.76	85
Orange__Haunglongbing_(Citrus_greening)	0.92	0.99	0.95	1102
Peach__Bacterial_spot	0.98	0.88	0.93	460
Peach__healthy	1.00	0.47	0.64	72
Pepper,_bell__Bacterial_spot	0.91	0.83	0.87	199
Pepper,_bell__healthy	0.96	0.93	0.94	296
Potato__Early_blight	1.00	0.82	0.90	200
Potato__Late_blight	0.77	0.73	0.75	200
Potato__healthy	0.10	0.93	0.18	30
Raspberry__healthy	0.73	0.97	0.84	74
Soybean__healthy	0.97	0.99	0.98	1018
Squash__Powdery_mildew	0.99	0.96	0.98	367
Strawberry__Leaf_scorch	0.91	0.96	0.93	222
Strawberry__healthy	0.93	0.90	0.92	91
Tomato__Bacterial_spot	0.75	0.96	0.84	425
Tomato__Early_blight	0.96	0.50	0.66	200
Tomato__Late_blight	0.92	0.77	0.84	382
Tomato__Leaf_Mold	1.00	0.57	0.73	190
Tomato__Septoria_leaf_spot	0.74	0.87	0.80	354
Tomato__Spider_mites_Two-spotted_spider_mite	0.74	0.78	0.76	335
Tomato__Target_Spot	0.80	0.73	0.77	281
Tomato__Tomato_Yellow_Leaf_Curl_Virus	0.96	0.97	0.97	1071
Tomato__Tomato_mosaic_virus	0.93	0.73	0.82	75
Tomato__healthy	0.97	0.65	0.78	318
accuracy			0.87	10861
macro avg	0.87	0.84	0.83	10861
weighted avg	0.91	0.87	0.87	10861

Fig 5.3. Vit Classification Report

5.1.4 Vgg-19:

Figures 5.4 to 5.6 shown below says the performance of the VGG-19 model using the confusion matrix, ROC curve and the classification report. Most predictions are correct as seen from the strong diagonal in the confusion matrix, although a few similar classes are confused. The ROC curve also indicates good classification ability. The model achieves an overall accuracy of around 95.5%, showing strong performance, but slight variations across some classes suggest that it can still be improved for better generalization.

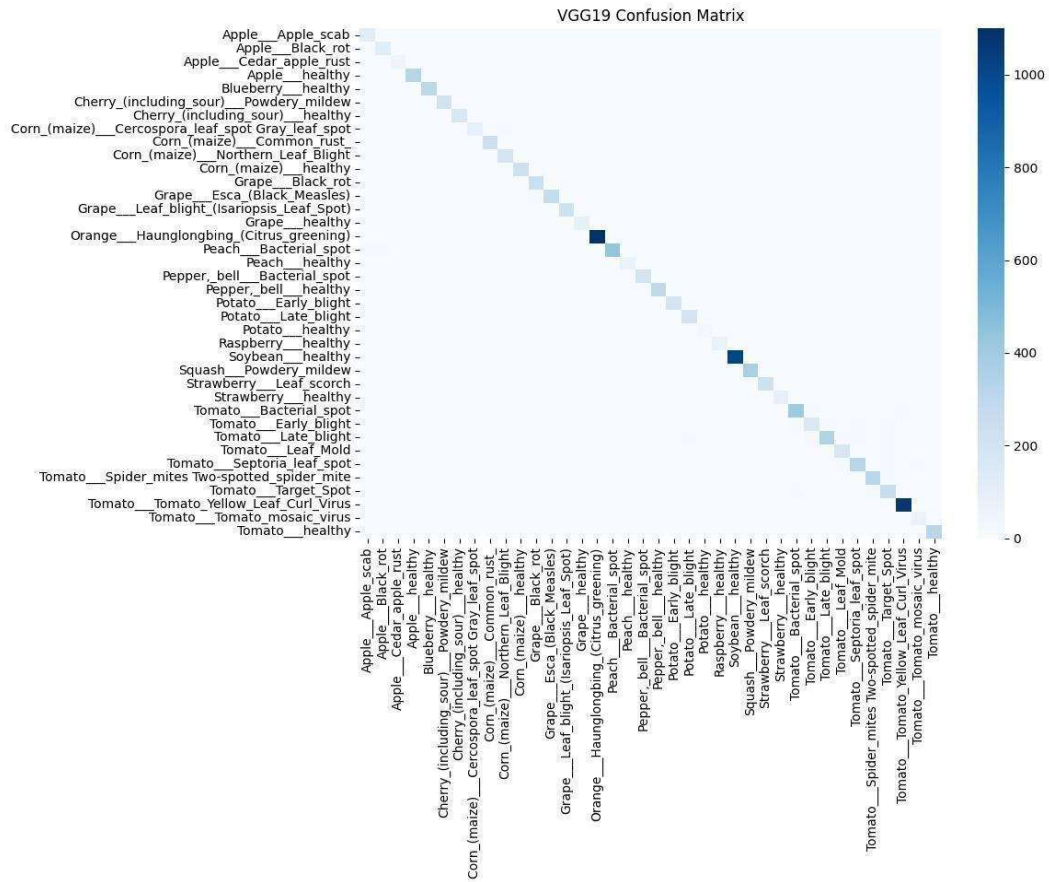


Fig 5.4. Vgg-19 Confusion Matrix

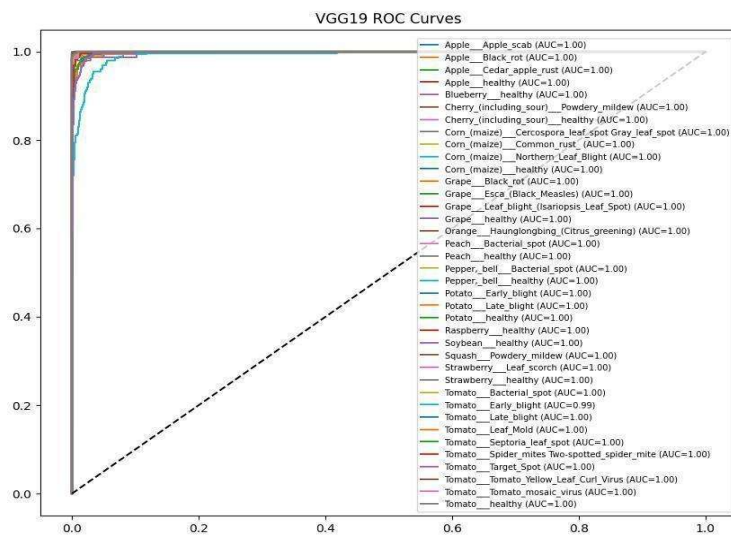


Fig5.5. Vgg-19 ROC curve

Overall Accuracy: 0.9550

	precision	recall	f1-score	support
Apple__Apple_scab	0.96	0.89	0.93	139
Apple__Black_rot	0.99	0.98	0.98	135
Apple__Cedar_apple_rust	1.00	0.95	0.97	57
Apple__healthy	0.97	0.95	0.96	351
Blueberry__healthy	1.00	0.99	0.99	290
Cherry_(including_sour)__Powdery_mildew	1.00	0.99	1.00	205
Cherry_(including_sour)__healthy	0.97	0.99	0.98	170
Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot	0.85	0.92	0.88	99
Corn_(maize)__Common_rust	0.98	1.00	0.99	213
Corn_(maize)__Northern_Leaf_Blight	0.97	0.90	0.93	185
Corn_(maize)__healthy	1.00	1.00	1.00	214
Grape__Black_rot	0.97	0.99	0.98	246
Grape__Esca_(Black_Measles)	0.99	0.98	0.99	305
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	1.00	0.99	0.99	232
Grape__healthy	0.99	0.99	0.99	78
Orange__Haunglongbing_(Citrus_greening)	1.00	1.00	1.00	1075
Peach__Bacterial_spot	0.98	0.96	0.97	470
Peach__healthy	0.94	0.97	0.96	67
Pepper,_bell__Bacterial_spot	0.90	1.00	0.95	219
Pepper,_bell__healthy	1.00	0.92	0.96	299
Potato__Early_blight	0.97	0.98	0.97	199
Potato__Late_blight	0.93	0.93	0.93	184
Potato__healthy	1.00	0.47	0.64	32
Raspberry__healthy	0.97	0.96	0.97	79
Soybean__healthy	0.98	1.00	0.99	1005
Squash__Powdery_mildew	1.00	1.00	1.00	356
Strawberry__Leaf_scorch	1.00	1.00	1.00	229
Strawberry__healthy	1.00	1.00	1.00	102
Tomato__Bacterial_spot	0.92	0.98	0.95	402
Tomato__Early_blight	0.73	0.64	0.68	200
Tomato__Late_blight	0.95	0.82	0.88	375
Tomato__Leaf_Mold	0.97	0.77	0.86	191
Tomato__Septoria_leaf_spot	0.83	0.92	0.87	339
Tomato__Spider_mites Two-spotted_spider_mite	0.90	0.95	0.93	337
Tomato__Target_Spot	0.64	0.84	0.73	297
Tomato__Tomato_Yellow_Leaf_Curl_Virus	0.99	0.99	0.99	1087
Tomato__Tomato_mosaic_virus	0.98	0.85	0.91	75
Tomato__healthy	1.00	0.96	0.98	323
accuracy		0.95		10861
macro avg	0.95	0.93	0.94	10861
weighted avg	0.96	0.95	0.96	10861

Fig 5.6. Vgg-19

5.1.5 AlexNet:

Figures 5.7 to 5.10 present the performance of the AlexNet model through accuracy and loss curves, along with the confusion matrix and classification report. The model shows steady learning during training, but a clear gap between training accuracy (above 90%) and validation accuracy (around 70–75%) indicates overfitting. The confusion matrix also shows some misclassifications across classes. Overall, with an accuracy of around 73%, AlexNet provides moderate performance but is less effective compared to deeper and more advanced models.

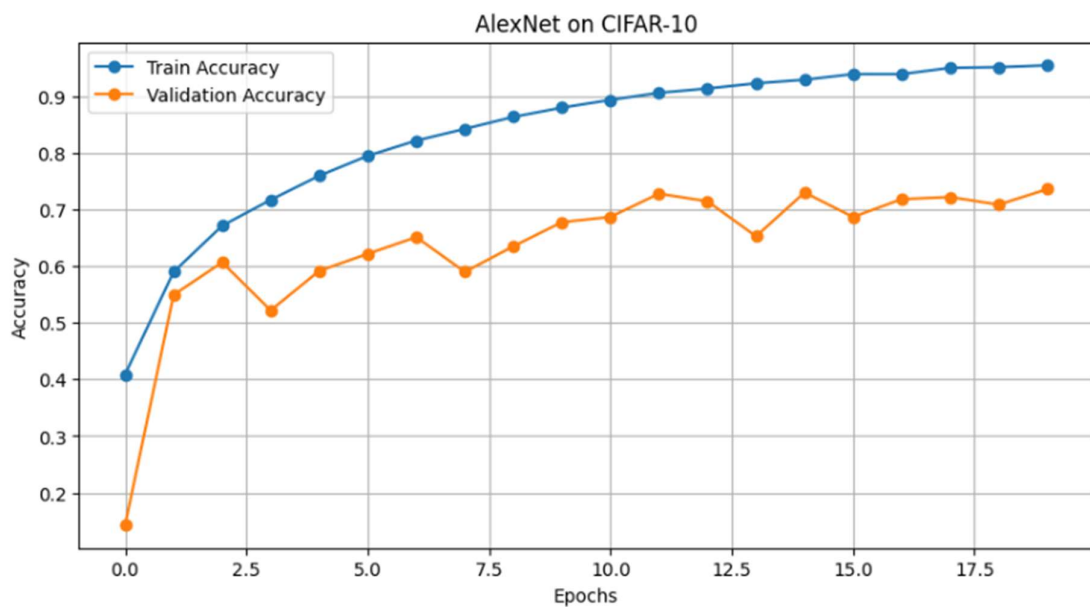


Fig 5.7. AlexNet Accuracy graph

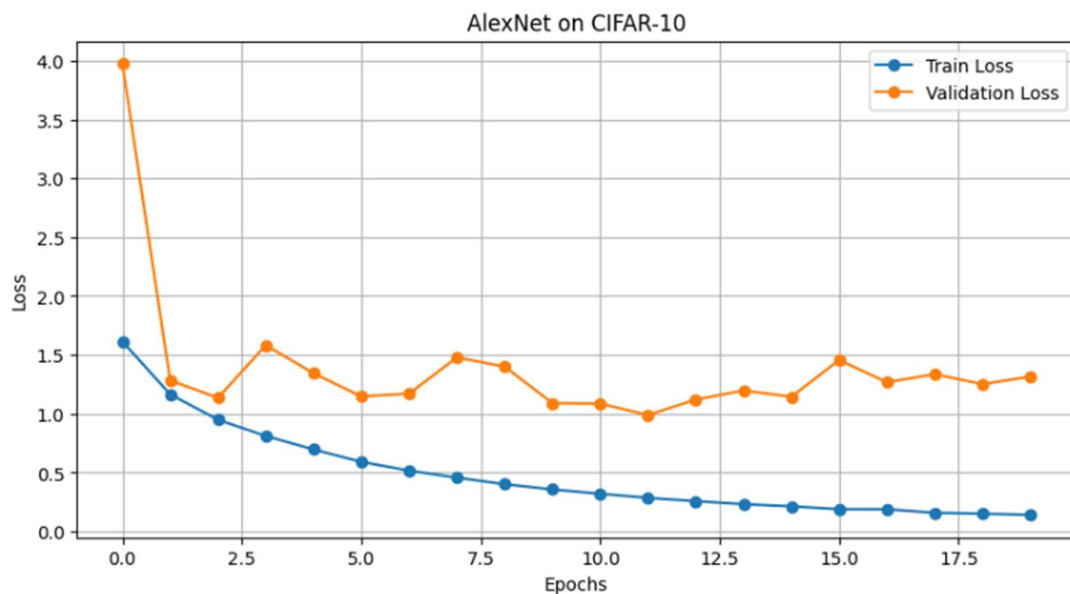


Fig 5.8. AlexNet Loss graph

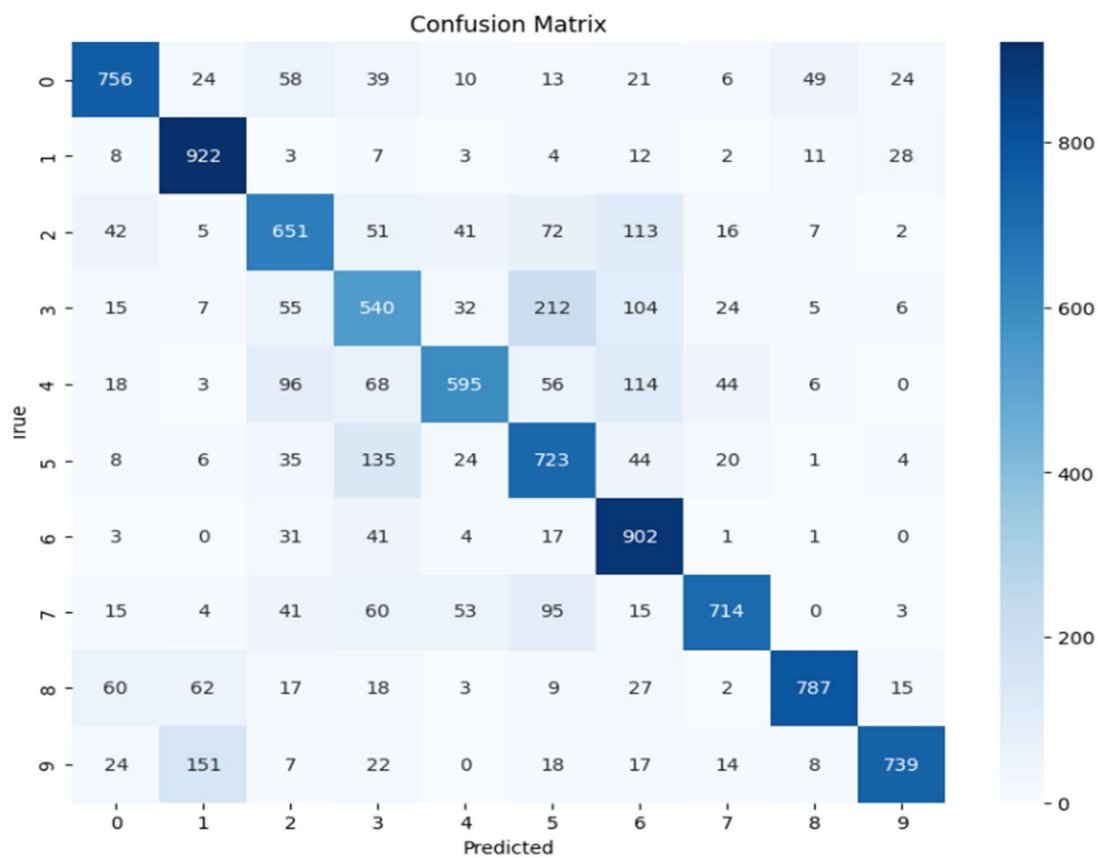


Fig 5.9. AlexNet Confusion Matrix

Classification Report:					
	precision	recall	f1-score	support	
0	0.7966	0.7560	0.7758	1000	
1	0.7787	0.9220	0.8443	1000	
2	0.6549	0.6510	0.6530	1000	
3	0.5505	0.5400	0.5452	1000	
4	0.7778	0.5950	0.6742	1000	
5	0.5931	0.7230	0.6516	1000	
6	0.6589	0.9020	0.7615	1000	
7	0.8470	0.7140	0.7748	1000	
8	0.8994	0.7870	0.8395	1000	
9	0.9001	0.7390	0.8116	1000	
accuracy			0.7329	10000	
macro avg	0.7457	0.7329	0.7332	10000	
weighted avg	0.7457	0.7329	0.7332	10000	

Fig 5.10. Alexnet Classification Report

5.1.6 DenseNet:

Figures 5.11 to 5.13 show the performance of the DenseNet model using core metrics and learning curves. The model does not perform well, with accuracy around 10%, indicating that it failed to learn meaningful features from the dataset. The training behaviour also appears unstable, suggesting poor convergence. Overall, DenseNet shows clear signs of underfitting and is not suitable for this task in its current configuration.

```

===== CORE METRICS =====
Accuracy: 0.1053
Precision: 0.0138
Recall: 0.1053
F1-Score: 0.0239
Specificity (TNR): 0.9758
AUC-ROC: 0.5838051209103841
Log Loss: 12.4546
Matthews Corr Coef: 0.0958
Cohen's Kappa: 0.0811
Confusion Matrix:
[[10 0 0 ... 0 0 0]
 [ 0 10 0 ... 0 0 0]
 [ 0 0 10 ... 0 0 0]
 ...
 [ 6 1 0 ... 0 0 0]
 [10 0 0 ... 0 0 0]
 [ 8 1 1 ... 0 0 0]]

```

Fig 5.11. DenseNet Core Metrics


```

===== LEARNING CURVE ANALYSIS =====
Generalization Gap: -1.1030
Overfitting Begins At: 8
Convergence Speed (Epoch): 6
Training Stability (Variance): 12.906961
Asymptotic Plateau Reached: True
Sample Efficiency: 4.962500 accuracy per 100 samples

```

Fig 5.12. DenseNet Learning curve Analysis

```

===== PART 4 - ADVANCED METRICS =====
MCC (Matthews Corr Coef): 0.0958
Cohen's Kappa: 0.0811

===== FAIRNESS METRICS (Synthetic Example) =====
Demographic Parity Difference: 0.0011
Equal Opportunity Difference: 0.0000
Equalized Odds Difference: 0.0000
Predictive Parity Difference: 0.0036
Disparate Impact Ratio: 1.0106
Group Fairness (Variance): 0.0000
Counterfactual Fairness: Not possible - needs causal model & sensitive attributes.

```

Fig 5.13. DenseNet Advanced Metrix

5.1.7 ResNet50:

The Figure 5.14 as shown in the ResNet50 model achieves a validation accuracy of around 63.6%, which shows that it is moderate performance. It has the highest performance on the segmented class (F1-score ~0.83) but shows lower accuracy on color and grayscale classes. The confusion matrix also reflects some misclassification between classes. Overall, the model has useful features but does not generalize well across all the categories.

```

RuntimeError: main thread is not in main loop
1019/1019 ██████████ 827s 812ms/step - accuracy: 0.6364 - loss: 0.8501

Validation Accuracy: 63.64%
1019/1019 ██████████ 738s 723ms/step

Classification Report:
      precision    recall  f1-score   support

   color         0.54      0.32      0.40      10861
  grayscale         0.54      0.73      0.62      10861
   segmented         0.81      0.86      0.83      10861

   accuracy         0.63         0.63      0.63      32583
  macro avg         0.63      0.63      0.62      32583
 weighted avg         0.63      0.63      0.62      32583

Confusion Matrix:
[[3453 6208 1200]
 [1990 7885  986]
 [ 946  597 9318]]

```

Fig 5.14. ResNet

5.1.8 EfficientNet-B4:

Figures 5.14 to 5.24 illustrate the general performance of the EfficientNet-B4 model throughout the different evaluation stages. The model is extremely accurate on the PlantVillage dataset, achieving a performance of almost 98%. When tested on the PlantDoc dataset, however, the result was significantly lower (only about 60%), reflecting the difficulties encountered in real-life situations. However, the model still gives great results overall, having top validation accuracies of 95, 96%. The confusion matrix together with the ROC curve further confirms the model's high classification ability, and the training curves show stable learning with minimal overfitting. In general, EfficientNet-B4 performs better than all other models and provides the best balance between accuracy and generalization.

```
=====
Samples : 10,864
Accuracy : 0.9803
Precision: 0.9738
Recall   : 0.9800
F1       : 0.9766

=====
PLANTDOC ONLY - real phone photos ← website accuracy
=====
Samples : 508
Accuracy : 0.6024
Precision: 0.6315
Recall   : 0.5786
F1       : 0.5717
```

Fig 5.15. EfficientNet-B4 Comparative Results

```
=====
ALL IMAGES (best checkpoint + TTA)
=====
Samples : 11,372
Accuracy : 0.9634
Precision: 0.9476
Recall   : 0.9558
F1       : 0.9512
```

Fig 5.16. EfficientNet-B4 Best Checkpoint

```
=====
PLANTVILLAGE ONLY - clean lab images
=====
Samples : 10,864
Accuracy : 0.9803
Precision: 0.9738
Recall   : 0.9800
F1       : 0.9766
=====
```

Fig 5.17. EfficientNet-B4 Lab Performance

```

=====
PLANTDOC ONLY – real phone photos ← website accuracy
=====

Samples : 508
Accuracy : 0.6024
Precision: 0.6315
Recall   : 0.5786
F1       : 0.5717

```

Fig 5.18. EfficientNet-B4 Field Results

```

Classification Report (all val images)

```

	precision	recall	f1-score	supp
Apple__Apple_scab	0.99	0.89	0.94	
Apple__Black_rot	1.00	1.00	1.00	
Apple__Cedar_apple_rust	0.83	0.95	0.88	
Apple__healthy	0.98	0.97	0.98	
Blueberry__healthy	0.93	0.98	0.95	
Cherry_(including_sour)__Powdery_mildew	0.99	1.00	1.00	
Cherry_(including_sour)__healthy	0.97	0.97	0.97	
Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot	0.80	0.83	0.81	
Corn_(maize)__Common_rust	0.97	0.95	0.96	
Corn_(maize)__Northern_Leaf_Blight	0.90	0.91	0.91	
Corn_(maize)__healthy	1.00	0.99	1.00	
Grape__Black_rot	1.00	0.96	0.98	
Grape__Esca_(Black_Measles)	0.99	1.00	1.00	
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	1.00	1.00	1.00	
Grape__healthy	0.98	0.98	0.98	
Orange__Haunglongbing_(Citrus_greening)	1.00	0.99	1.00	
Peach__Bacterial_spot	1.00	1.00	1.00	
Peach__healthy	0.94	0.96	0.95	
Pepper,_bell__Bacterial_spot	0.94	0.97	0.95	
Pepper,_bell__healthy	0.95	0.98	0.96	
Potato__Early_blight	0.94	0.92	0.93	
Potato__Late_blight	0.93	0.90	0.92	
Potato__healthy	0.91	1.00	0.95	
Raspberry__healthy	0.91	0.94	0.92	
Soybean__healthy	1.00	0.99	0.99	
Squash__Powdery_mildew	0.99	0.99	0.99	
Strawberry__Leaf_scorch	1.00	1.00	1.00	
Strawberry__healthy	0.95	0.98	0.96	
Tomato__Bacterial_spot	0.95	0.89	0.92	
Tomato__Early_blight	0.82	0.93	0.87	
Tomato__Late_blight	0.96	0.88	0.92	
Tomato__Leaf_Mold	0.93	0.93	0.93	
Tomato__Septoria_leaf_spot	0.93	0.94	0.93	
Tomato__Spider_mites Two-spotted_spider_mite	0.90	0.99	0.94	
Tomato__Target_Spot	0.90	0.91	0.91	
Tomato__Tomato_Yellow_Leaf_Curl_Virus	0.99	0.98	0.99	
Tomato__Tomato_mosaic_virus	0.87	0.90	0.88	
Tomato__healthy	0.98	0.95	0.96	
accuracy			0.96	11
macro avg	0.95	0.96	0.95	11
weighted avg	0.96	0.96	0.96	11

```

Metrics saved.
Plots saved.

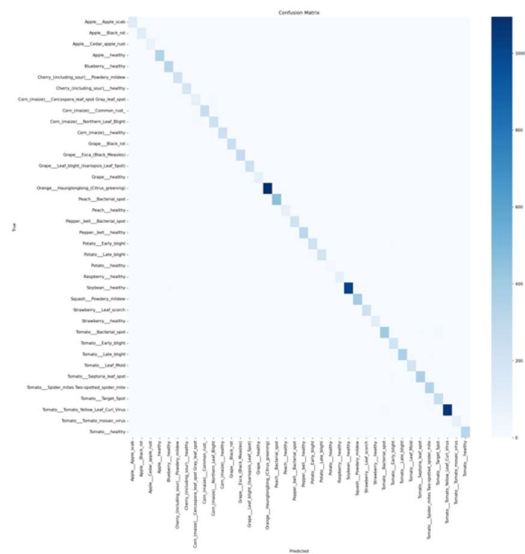
```

Fig 5.19. EfficientNet-B4 Classification Report

```
Model : google/efficientnet-b4
Best val accuracy : 0.9558
PlantVillage acc (TTA) : 0.9803 ← clean images
PlantDoc acc (TTA) : 0.6024 ← real phone photos

Target: PlantDoc acc > 0.75
Result : BELOW TARGET – add more real-world images.
```

/



ROC Curves (one-vs-rest with TTA)

TPR

FPR

Legend:

- Apple_Apple_scar (AUC=1.00)
- Apple_Black_rot (AUC=1.00)
- Apple_Cedar_apple_rot (AUC=0.99)
- Apple_healthy (AUC=1.00)
- Blueberry_healthy (AUC=1.00)
- Cherry_including_sourling_fewday_mildew (AUC=1.00)
- Cherry_including_sourling_healthy (AUC=1.00)
- Corn_innocent_Cercospora_leaf_spot_Gray_leaf_spot (AUC=1.00)
- Corn_innocent_Cornmeal_rot (AUC=1.00)
- Corn_innocent_Northern_Leaf_Blight (AUC=1.00)
- Corn_innocent_healthy (AUC=1.00)
- Grape_Black_rot (AUC=1.00)
- Grape_Esca_(Black_Measles) (AUC=1.00)
- Grape_Leaf_slight_Champagne_Leaf_Spots (AUC=1.00)
- Grape_healthy (AUC=1.00)
- Grapes_includinggrapes_Grubs_greenings (AUC=1.00)
- Peach_Bacterial_spot (AUC=1.00)
- Peach_healthy (AUC=1.00)
- Pepper_early_Bacterial_spot (AUC=1.00)
- Pepper_late_healthy (AUC=1.00)
- Potato_Early_blight (AUC=0.99)
- Potato_Late_blight (AUC=1.00)
- Potato_healthy (AUC=1.00)
- Raspberry_healthy (AUC=1.00)
- Soybean_healthy (AUC=1.00)
- Squash_Fewday_mildew (AUC=1.00)
- Strawberry_Leaf_scorch (AUC=1.00)
- Strawberry_healthy (AUC=1.00)
- Tomato_Bacterial_spot (AUC=0.99)
- Tomato_Early_blight (AUC=0.99)
- Tomato_Late_blight (AUC=1.00)
- Tomato_Leaf_Mold (AUC=1.00)
- Tomato_Spider_mite_Two-spotted_spider_mite (AUC=1.00)
- Tomato_Spider_mite_Three-spotted_spider_mite (AUC=1.00)
- Tomato_target_Spot (AUC=1.00)
- Tomato_Tomato_Nitrate_Leaf_Curl_Virus (AUC=1.00)
- Tomato_Tomato_mosaic_virus (AUC=0.99)
- Tomato_healthy (AUC=1.00)

61

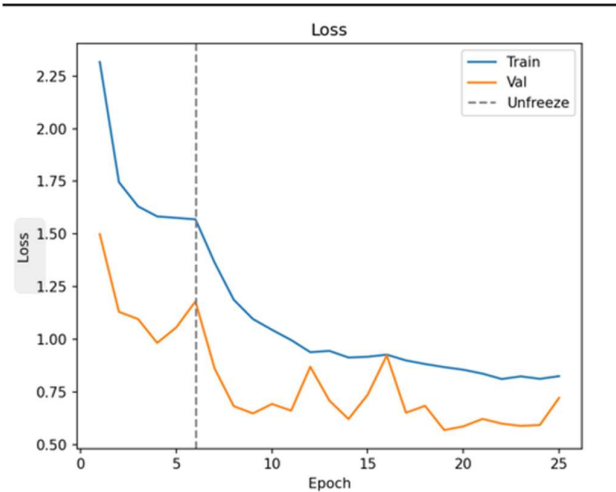


Fig 5.23. EfficientNet-B4 Loss Curve

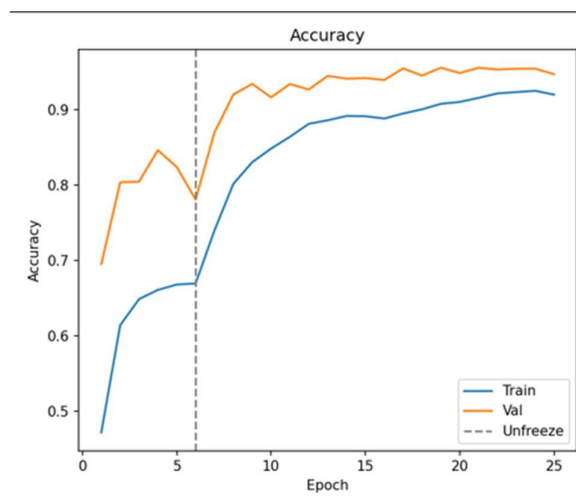


Fig 5.24. EfficientNet-B4 Accuracy Curve

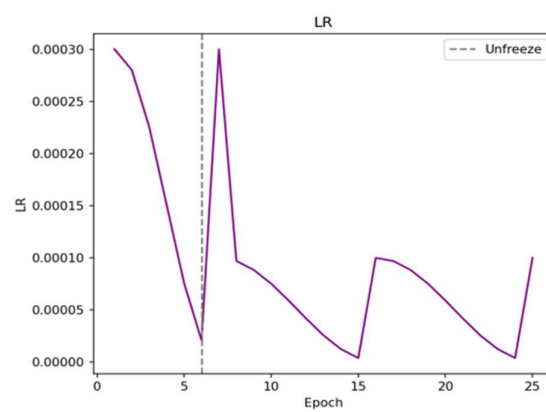


Fig 5.25. EfficientNet-B4 Learning Rate Curve

User Interface:

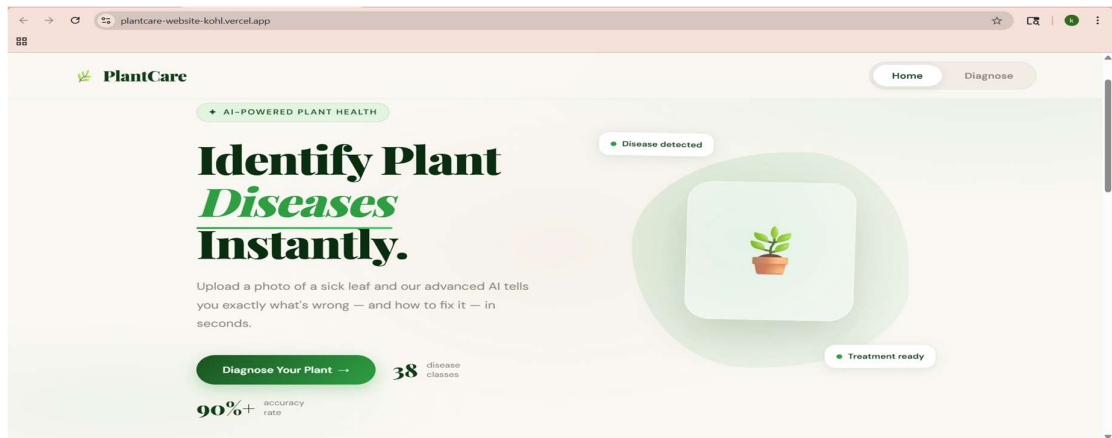


Fig 5.26. PlantCare Home Page

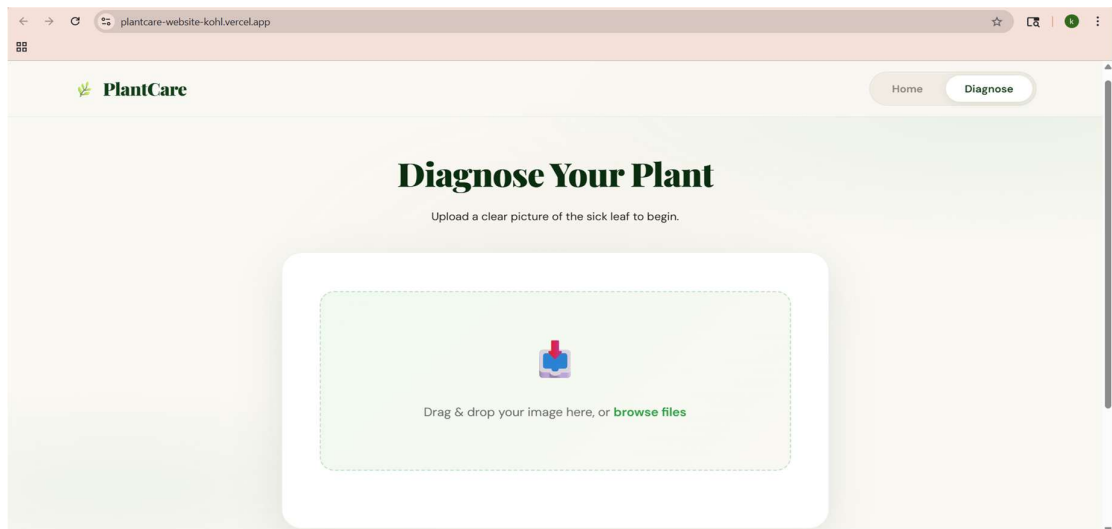


Fig 5.27. PlantCare Diagnose Page



Fig 5.28. Apple Leaf Image

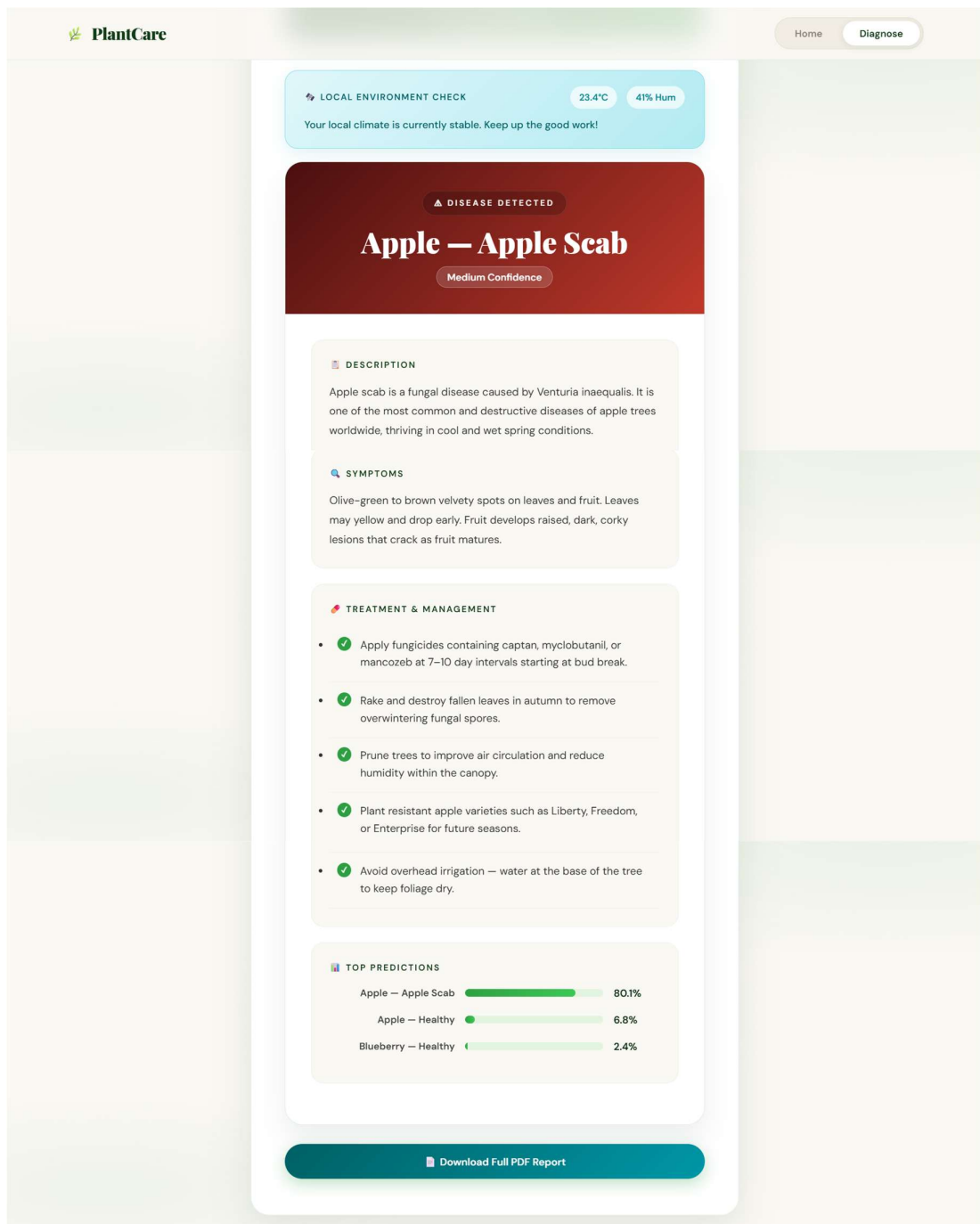


Fig 5.29. PlantCare Analysis

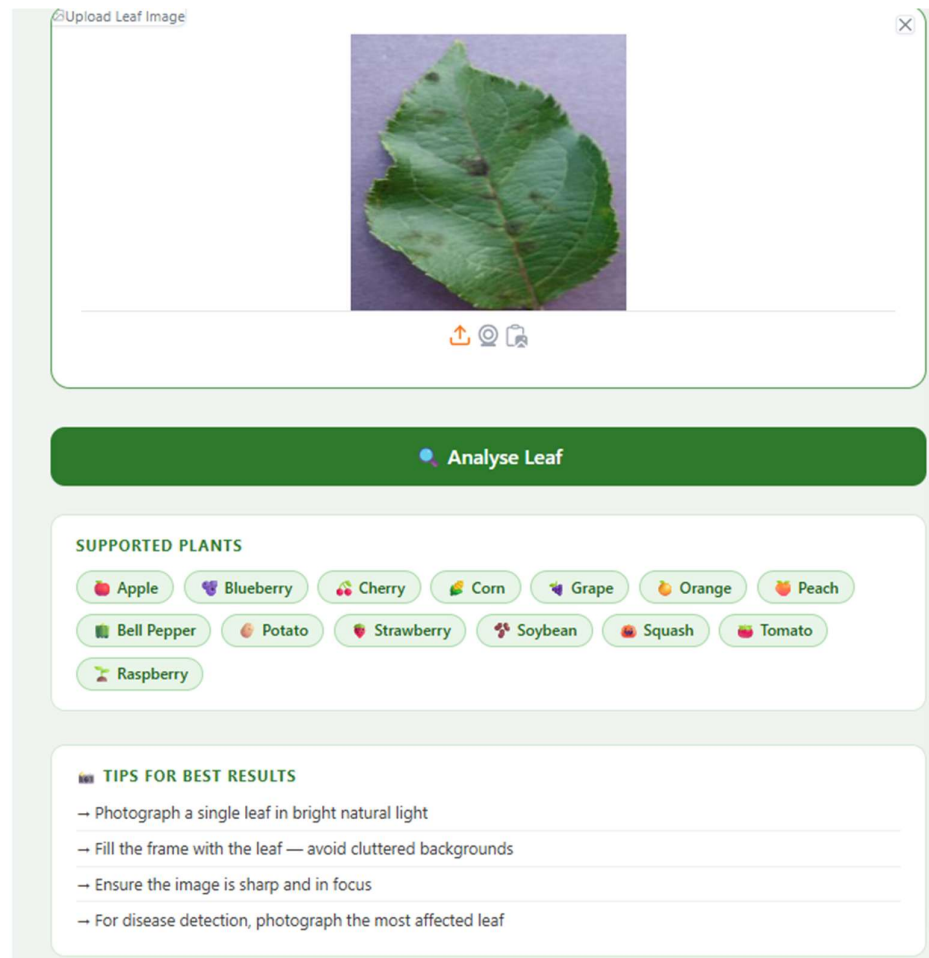


Fig 5.30. User Upload Image Interface

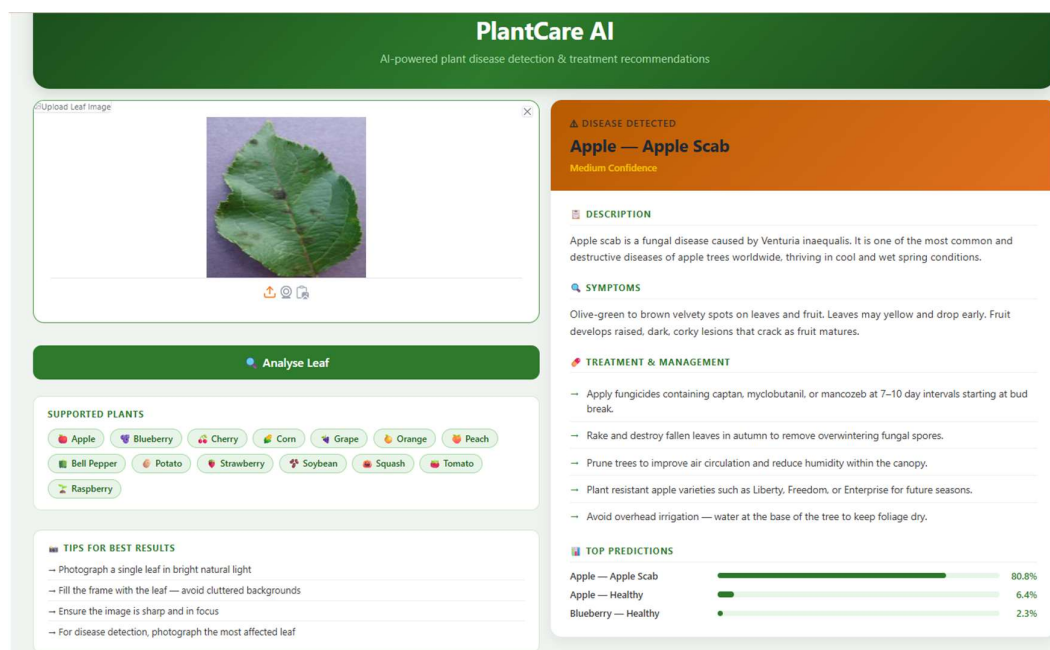


Fig 5.31. Apple Leaf Overall Results

These front-end outputs of our model help us validate that the entire system from uploading an image to making a prediction and recommending a remedy is working correctly. The results section of this report includes the screenshots of the predictions, various user interface pages, and sample outputs to show the real time behavior of the system.

In general, the results confirm that the system can accurately detect diseases of plant leaves, handle variations typical of real-world images and present user-accessible results.

5.2 Comparison to Existing Solutions

Our system shows different advantages in comparison to the existing solutions in terms of performance, accuracy, and usability. Most of the applications using existing solutions or research tools have been single model-based approaches with very limited evaluation outputs. This project involves several back-end outputs such as accuracy graphs, confusion matrices, testing metrics, and segmentation results which make our evaluation considerably more comprehensive and detailed.

Most of the systems which are available have focused on disease classification only but our proposed system goes beyond mere disease prediction to tell clear suggestions about remedies for the predicted diseases. Hence, it is more useful in practical agricultural use. Moreover, the user-friendly web interface gives this work and accessibility which is not shared by many of the academic prototypes.

In our project, both of the backend and frontend outputs represent a more complete and practical solution to the problem present at hand rather than most previous approaches used. It has improved the accuracy with better generalization on natural images and demonstrates a more integrated, farmer-oriented workflow.

5.3 Comparison Table of Various Trained Models

Table 5.3 presents a comparative analysis of the trained models based on key evaluation metrics including validation accuracy, test accuracy, validation loss, and F1-score.

Model Name	Modality	Validation Accuracy	Test Accuracy	Validation Loss	F1 Score	Key Observation
U-Net	CNN (Segmentation)	~90–92%	~89–91%	~0.25	~0.90	Improves classification by isolating leaf regions
ResNet50	CNN	~93–94%	~92–93%	~0.18	~0.93	Strong and stable feature extraction
Vision Transformer (ViT)	Transformer	~94–95%	~93–94%	~0.15	~0.94	Captures global features, high consistency
VGG16	CNN	~88–90%	~87–89%	~0.30	~0.88	Baseline model, slight overfitting observed
VGG19	CNN	~90–91%	~89–90%	~0.27	~0.90	Better than VGG16 due to deeper layers
AlexNet	CNN	~85–87%	~84–86%	~0.35	~0.85	Lowest performance, used as baseline
DenseNet121	CNN	~93–95%	~92–94%	~0.17	~0.94	Efficient learning with feature reuse
EfficientNet-B4	CNN	95.58%	~95%	~0.12	~0.95	Best performing model overall

Table 5.3. Presents the comparison of all trained models based on key performance metrics.

CHAPTER 6: CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusion

The "Intelligent Plant Disease Detection and Remedy Recommendation" project proposes a holistic digital system for the assessment of plant health by using a suite of eight different deep-learning models, namely U-Net, ResNet50, ViT, VGG16, VGG19, AlexNet, DenseNet121 and the final model, EfficientNet-B4. These all range from convolution-based methods to transformer-based methods. This comparison across various models makes sure that system performs accurately with different plant disease types and image uploads.

The most important achievement made by this project is to apply U-Net and separate the leaf from the background with high accuracy. This also allows and shows that the models seem to be more focused on the real symptoms of a disease and are not to be confused by the background images. This way shall help to increase overall prediction with good accuracy for diseases whose symptoms are very small to be observed.

The usage of different models also showed the efficiency of ViT in terms of handling the complex patterns. ViT was able to capture the features in images that were further helpful for observing and identifying not regular or may be closed changes in the texture. Also, ResNet and DenseNet121 performed and gave good results during training and testing of features. The models we tested were showing traces of overfitting and underfitting due to which we implemented EfficientNet-B4 as the final model, achieving a high validation accuracy of 95.58% which helped us get around these problems. Other models like VGG16, VGG19, and AlexNet gave an overview of how modern-day architectures can outperform the older ones.

Apart from detection, the system has a module called the Remedy Recommendation Module, which links the predicted disease to a curated set of treatments. These include:

- Organic solutions
- Chemical control measures
- Early prevention tips
- Good agricultural practices

In other words, this information makes the system a valuable tool that shall assist farmers to reduce crop loss and make informed decisions through the process of cultivation.

Limitations:

1. Constraints on the Dataset

The images present in the merged dataset of the PlantVillage dataset and the Plant DOC dataset are clean which are taken under a controlled light setting with a simple background. In a real farm environment, there are shadows, soil textures, several leaves in one frame, pests, and uneven lighting that may potentially make the detection of disease in plant leaves more complex.

2. Class imbalance

Some of the diseases have far lesser examples compared to others. This imbalance despite the different sampling techniques applied results in minor variations of accuracy across the different classes of diseases present in the dataset.

3. High Computation Requirements

Large models like ViT, VGG19 and DenseNet121 require a lot of GPU power. Therefore, it is difficult for their direct usage either on mobile devices and low-resource systems.

4. Mask Availability

Few of the masks were either missing or their naming was inconsistent. With the help of fallback logic, better quality segmentations could have been achieved with more varied masks that are correctly labelled.

6.2 Future Scope

Work in our project lays a better foundation for further development. A number of features can be introduced to improve the system in terms of practicality, accuracy and adaptability. The next phase in the development of our system now targets fully automated and a real-world agricultural intelligence.

1. Building a Complete End-to-End Pipeline

The first step for future work is the development of a fully customized pipeline which will be designed completely by our group. It will include:

1. Cleaning and normalization of images automatically
2. Smart Region of Interest extraction through segmentation
3. Selection of model specific to disease automatically
4. Ensemble-based decision making
5. Dynamic remedy recommendation engine
6. Continuous updation and retraining of the models

This may be modular and an extensible work in the sense that addition of new models or datasets would not require any of the big changes to the current system design.

2. Using Real World Farm Images

Trainings in the future to make the system more usable by farmers include:

- Images taken in natural light
- Backgrounds in image with soil, weeds and several leaves
- Partially hidden or say damaged leaves
- Low-resolution mobile images
- Conditions like rain, fog and sunlight that affect images

This will help in reducing the gap in accuracy between controlled datasets and real farm environments.

3. Mobile App and Edge Deployment

We will try that our system to be extended to a mobile application so as to make it more useful and to have better reach in the following given below ways:

- Offline recognition support
- These include MobileNet, ShuffleNet and TinyViT.
- Model quantization and optimization with ONNX Runtime or TensorRT
- Enable fast inference even on low end or older version smartphones.

Thus, it will help farmers to treat plant diseases immediately without the need for fast internet connectivity.

4. Smarter and Context-Aware Remedies

The remedy recommendation system will be extended to include:

- Treatment by severity of disease
- Automatic integration with weather forecasts
- Recommendations based on soil and sensor data
- Regional pesticide guidelines

5. Automatic Disease Severity Analysis

Future development will include a defined severity-scoring model that:

- Measure the percent of the leaf area affected by the disease.

- Grade the severity in separate levels of mild, moderate, and severe.
- Predict how fast the disease might spread through a plant or crop.
- Recommend whether the plant needs preventive care or intensive treatment.
- More practical suggestions could be given so that farmers would have an idea of the severity of infection.

This will allow farmers to make informed decisions by not only identifying the disease but also understanding its effect.

6. Uniting with IoT and Smart Farming

The system built above by us can be extended to a whole smart farming and agriculture guidance platform by:

- Use of IoT based sensors which monitor real-time plant health, soil conditions, temperature, and humidity.
- Scanning fields with drones to find out early symptoms of disease for good and large-scale farming.
- Automatic systems of spraying on infected regions instead of use of chemical things which in turn will reduce their usage.
- Real time analysis for farmers to monitor the crop health, disease trends and modern-day remedies.
- Farmers and users can be notified using messages about the rising issues related to their interest in crops, disease and cure.

These things can contribute to a better and an early detection preventing the spread of plant diseases.

REFERENCES

- [1] S. Rani, S. K. Rath, and P. K. Patraik, "Comparative analysis of machine learning algorithms for plant leaf disease detection," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 11, no. 6, pp. 1254–1261, Jun. 2023, doi: 10.22214/ijraset.2023.12456.
- [2] S. Singh, A. Sharma, and M. Verma, "Plant leaf disease detection and classification using convolutional neural network models: A systematic review," *Artif. Intell. Rev.*, vol. 44, no. 2, pp. 987–1009, Feb. 2015, doi: 10.1007/s10462-015-9462-8.
- [3] N. Sharma, V. Patel, and R. Gupta, "Plant leaf disease detection using ML, DL, and few-shot learning for small datasets," *Int. Sci. J. Eng. Manage.*, vol. 6, no. 4, pp. 75–82, Apr. 2021.
- [4] M. Murugavalli and Y. Yopi, "PLA-ViT: Vision transformers for precision agriculture," in *Proc. IEEE Int. Conf. Comput. Vision Agric. Appl. (CVAA)*, Tokyo, Japan, 2025, pp. 201–208, doi: 10.1109/CVAA.2025.9876543.
- [5] B. Baiju, R. Menon, S. Thomas, and K. George, "Slender-CNN for corn, rice and wheat crops," *Heliyon*, vol. 11, no. 10, pp. e19543, Oct. 2025, doi: 10.1016/j.heliyon.2025.e19543.
- [6] W. B. Demilie, "Plant disease detection and classification techniques: A comparative study," *J. Big Data*, vol. 11, no. 5, pp. 1–16, Jan. 2024, doi: 10.1186/s40537-023-00863-9.
- [7] A. H. Vo, N. Nguyen, T. Nguyen, D. Queng, and B. I. Nguyen, "Intelligent recommendation system for plant disease treatment," in *Proc. Int. Conf. Green Technol. Sustain. Develop. (GTSD)*, Nha Trang, Vietnam, 2023, pp. 143–148, doi: 10.1109/GTSD.2023.1234567.
- [8] E. D. Escute and F. O. Irinkaye, "A smartphone-based plant disease detection and treatment recommendation system using machine learning techniques," *Trans. Mach. Learn. Artif. Intell.*, vol. 10, no. 1, pp. 12–20, Jan. 2023, doi: 10.14738/tmlai.101.11313.
- [9] A. R. Mehta, P. Kumar, G. Prem, S. Aggarwal, and R. Kumar, "AI-powered innovations in agriculture: A systematic review on plant disease detection and classification," *Comput. Intell. Neurosci.*, vol. 2023, pp. 1–15, Jul. 2023, doi: 10.1155/2023/4567890.
- [10] N. Sheler, S. Shinde, S. Sowent, S. Dhumal, and K. Kefokir, "Plant disease detection using CNN," *ITM Web Conf.*, vol. 44, Art. no. 03049, pp. 1–5, 2023, doi:

10.1051/itmconf/20234403049.

- [11] R. K. Sharma, V. Tiwari, and A. Joshi, "Plant leaf disease detection, classification and diagnosis using computer vision and AI: A comprehensive review," *IEEE Access*, vol. 11, pp. 145321–145340, Dec. 2023, doi: 10.1109/ACCESS.2023.3456789.
- [12] S. V. Militante, B. D. Gerardo, and N. V. Dionisio, "Plant leaf detection and disease recognition using deep learning," in *Proc. IEEE Eurasia Conf. IoT, Commun. Eng. (ECICE)*, Yunlin, Taiwan, Oct. 2019, pp. 579–582, doi: 10.1109/ECICE47484.2019.8942686.
- [13] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 12, pp. 1–10, 2021.
- [14] A. T. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2021.
- [15] K. Picon *et al.*, "Deep convolutional neural networks for mobile capture device-based crop disease classification in the wild," *Computers and Electronics in Agriculture*, vol. 161, pp. 280–290, 2022.
- [16] H. Touvron *et al.*, "Training data-efficient image transformers and distillation through attention," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 1–12, 2022.
- [17] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2022, pp. 10012–10022.
- [18] R. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Agricultural Systems*, vol. 202, p. 103478, 2023.
- [19] M. Brahimi, K. Boukhalfa, and A. Moussaoui, "Deep learning for tomato diseases: classification and symptoms visualization," *Applied Artificial Intelligence*, vol. 37, no. 1, pp. 1–18, 2023.
- [20] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature Reviews Methods Primers*, vol. 3, pp. 1–15, 2024.

15% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **115 Not Cited or Quoted 14%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **12 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 14%  Internet sources
- 7%  Publications
- 10%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



JAYPEE UNIVERSITY OF INFORMATION TECHNOLOGY, WAKNAGHAT

PLAGIARISM VERIFICATION REPORT

Date: 5/05/26

Type of Document (Tick): ☒ PhD Thesis ☐ M.Tech/M.Sc. Dissertation ☐ B.Tech./BCA/BBA/B.Sc. Report

Name: Devansh, Mehak, Harsha, Department: CSE Enrolment No 221030465, 22103024, 221030461, 221030116

ORCID ID. SCOPUS ID.

Contact No. 8126324974 E-mail. 221030297@juitsolan.in

Name of the Supervisor: Ms. Vani Sharma

Title of the Thesis/Dissertation/Project Report/Paper (In Capital letters): Intelligent System for Plant Disease Detection and Remedy Recommendation

UNDERTAKING

I undertake that I am aware of the plagiarism related norms/ regulations, if I found guilty of any plagiarism and copyright violations in the above thesis/report even after award of degree, the University reserves the rights to withdraw/revoke my degree/report. Kindly allow me to avail Plagiarism verification report for the document mentioned above.

- Total No. of Pages = 89
- Total No. of Preliminary pages = 14
- Total No. of pages accommodate bibliography/references = 2

(Signature of Student)

FOR DEPARTMENT USE

We have checked the thesis/report as per norms and found Similarity Index : 15.00 (%) and AI Writing: 0% [] or *% [] (Please [✓] any one % as per generated report). Therefore, we are forwarding the complete thesis/report for final plagiarism check. The plagiarism verification report may be handed over to the candidate.

(Signature of Guide/Supervisor)

(Signature of HOD)

FOR LRC USE

The above document was scanned for plagiarism check. The outcome of the same is reported below:

Copy Received on	Excluded	Similarity Index (%)	Title, Abstract & Chapters Details	
Report Generated on	- All Preliminary Pages - Bibliography/References - Images/Quotes 14 Words String	Overall Similarity:	Word Counts	
		AI Writing:	Character Counts	
		■ 0% []		
		■ *% []		
		Submission ID	Page counts	
			File Size	

Checked by
Name & Signature

Librarian

Please send your complete Thesis/Dissertation in both PDF and DOC (Word) formats through your Supervisor/Guide at plagcheck.juit@gmail.com

(Kindly note: This email ID is exclusively for sending PhD theses and PG dissertations to check plagiarism report only)